



中山大學
SUN YAT-SEN UNIVERSITY

《计算机组成原理实验》 实验报告

(实验三)

学 院 名 称 : 计算机学院
专业 (班级) : 22 计科教学三班
学 生 姓 名 :
学 号 :
时 间 : 2023 年 1 月 22 日

成绩：

实验三：多周期CPU设计与实现

一. 实验目的

1. 认识和掌握多周期数据通路图的构成、原理及其设计方法。
2. 掌握多周期CPU的实现方法，代码实现方法。
3. 编写一个编译器，将 MIPS 汇编程序编译为二进制机器码。
4. 掌握多周期CPU的测试方法。
5. 掌握多周期CPU的实现方法。

二. 实验内容

设计一个多周期CPU，需要设计的指令与格式如下：

==> 算术运算指令

(1) add rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100000
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] + GPR[rt]。

(2) sub rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100010
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] - GPR[rt]。

(3) addiu rt, rs, immediate

001001	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] + sign_extend(immediate); immediate 做符号扩展再参加“与”运算。

==> 逻辑运算指令

(4) andi rt, rs, immediate

001100	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] and zero_extend(immediate); immediate 做 0 扩展再参加“与”运算。

(5) and rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100100
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] and GPR[rt]。

(6) ori rt, rs, immediate

001101	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] or zero_extend(immediate)。

(7) or rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100101
--------	---------	---------	---------	--------------

功能: $\text{GPR}[\text{rd}] \leftarrow \text{GPR}[\text{rs}] \text{ or } \text{GPR}[\text{rt}]$ 。

(8) xori rt, rs, immediate

001110	rs(5 位)	rt(5 位)	immediate(16 位)
--------	---------	---------	-----------------

功能: $\text{GPR}[\text{rt}] \leftarrow \text{GPR}[\text{rs}] \text{ XOR } \text{zero_extend}(\text{immediate})$ 。

==>移位指令

(9) sll rd, rt, sa

000000	00000	rt(5 位)	rd(5 位)	sa(5 位)	000000
--------	-------	---------	---------	---------	--------

功能: $\text{GPR}[\text{rd}] \leftarrow \text{GPR}[\text{rt}] \ll \text{sa}$ 。

==>比较指令

(10) slti rt, rs, immediate 带符号数

001010	rs(5 位)	rt(5 位)	immediate(16 位)
--------	---------	---------	-----------------

功能: if $\text{GPR}[\text{rs}] < \text{sign_extend}(\text{immediate})$ $\text{GPR}[\text{rt}] = 1$ else $\text{GPR}[\text{rt}] = 0$ 。

(11) slt rd, rs, rt 带符号

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 101010
--------	---------	---------	---------	--------------

功能: if $(\text{GPR}[\text{rs}] < \text{GPR}[\text{rt}])$ $\text{GPR}[\text{rd}] = 1$ else $(\text{GPR}[\text{rd}] = 0)$ 。

==> 存储器读/写指令

(12) sw rt, offset(rs) 写存储器

101011	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

功能: $\text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})] \leftarrow \text{GPR}[\text{rt}]$ 。

(13) lw rt, offset(rs) 读存储器

100011	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

功能: $\text{GPR}[\text{rt}] \leftarrow \text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})]$ 。

==> 分支指令

(14) beq rs, rt, offset

000100	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

功能: if $(\text{GPR}[\text{rs}] = \text{GPR}[\text{rt}])$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$

特别说明: offset 是从 PC+4 地址开始和转移到的指令之间指令条数。offset 符号扩展之后左移 2 位再相加。为什么要左移 2 位? 由于跳转到的指令地址肯定是 4 的倍数(每条指令占 4 个字节), 最低两位是“00”, 因此将 offset 放进指令码中的时候, 是右移了 2 位的, 也就是以上说的“指令之间指令条数”。

(15) bne rs, rt, offset

000101	rs(5 位)	rt(5 位)	offset(16 位)
--------	---------	---------	--------------

功能: if $(\text{GPR}[\text{rs}] \neq \text{GPR}[\text{rt}])$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$

(16) bltz rs, offset

000001	rs(5 位)	00000	offset(16 位)
--------	---------	-------	--------------

功能: if $(\text{GPR}[\text{rs}] < 0)$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$ 。

==>跳转指令

(17) j addr

000010	addr(26 位)
--------	------------

功能: $PC \leftarrow \{PC[31:28], \text{addr}, 2'b0\}$, 无条件跳转。

说明: 由于 MIPS32 的指令代码长度占 4 个字节, 所以指令地址二进制数最低 2 位均为 0, 将指令地址放进指令代码中时, 可省掉! 这样, 除了最高 6 位操作码外, 还有 26 位可用于存放地址, 事实上, 可存放 28 位地址, 剩下最高 4 位由 pc+4 最高 4 位拼接上。

(18) jr rs

000000	rs(5 位)	未用	未用	001000
--------	---------	----	----	--------

功能: $PC \leftarrow GPR[rs]$, 跳转。

==>调用子程序指令

(19) jal addr

000011	addr[27:2]
--------	------------

功能: 调用子程序, $PC \leftarrow \{PC[31:28], \text{addr}, 2'b0\}$; $GPR[\$31] \leftarrow pc+4$, 返回地址设置; 子程序返回, 需用指令 jr \$31。跳转地址的形成同 j addr 指令。

==> 停机指令

(20) halt

111111	000000000000000000000000(26 位)
--------	--------------------------------

功能: 停机; 不改变 PC 的值, PC 保持不变。

三. 实验原理

多周期 CPU 指的是将整个 CPU 的执行过程分成几个阶段, 每个阶段用一个时钟去完成, 然后开始下一条指令的执行, 而每种指令执行时所用的时钟数不尽相同, 这就是所谓的多周期 CPU。CPU 在处理指令时, 一般需要经过以下几个阶段:

(1) 取指令(IF): 根据程序计数器 pc 中的指令地址, 从存储器中取出一条指令, 同时, pc 根据指令字长度自动递增产生下一条指令所需要的指令地址, 但遇到“地址转移”指令时, 则控制器把“转移地址”送入 pc, 当然得到的“地址”需要做些变换才送入 pc。

(2) 指令译码(ID): 对取指令操作中得到的指令进行分析并译码, 确定这条指令需要完成的操作, 从而产生相应的操作控制信号, 用于驱动执行状态中的各种操作。

(3) 指令执行(EXE): 根据指令译码得到的操作控制信号, 具体地执行指令动作, 然后转移到结果写回状态。

(4) 存储器访问(MEM): 所有需要访问存储器的操作都将在这个步骤中执行, 该步骤给出存储器的数据地址, 把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

(5) 结果写回(WB): 指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。

实验中就按照这五个阶段进行设计, 这样一条指令的执行最长需要五个(小)时钟周期才能完成, 但具体情况怎样? 要根据该条指令的情况而定, 有些指令不需要五个时钟周期的, 这就是多周期的 CPU。

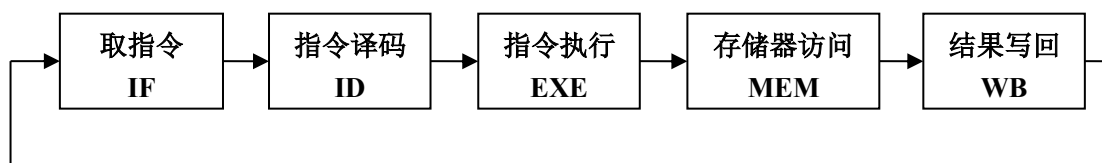


图 1 多周期 CPU 指令处理过程

MIPS 指令的三种格式:

R 类型:

31	26 25	21 20	16 15	11 10	6 5	0
op	rs	rt	rd	sa	funct	
6 位	5 位	5 位	5 位	5 位	6 位	

I 类型:

31	26 25	21 20	16 15	0
op	rs	rt	immediate	
6 位	5 位	5 位	16 位	

J 类型:

31	26 25	0
op	address	
6 位	26 位	

其中,

op: 为操作码;

rs: 为第 1 个源操作数寄存器, 寄存器地址 (编号) 是 00000~11111, 00~1F;

rt: 为第 2 个源操作数寄存器, 或目的操作数寄存器, 寄存器地址 (同上);

rd: 为目的操作数寄存器, 寄存器地址 (同上);

sa: 为位移量 (shift amt), 移位指令用于指定移多少位;

funct: 为功能码, 在寄存器类型指令中 (R 类型) 用来指定指令的功能;

immediate: 为 16 位立即数, 用作无符号的逻辑操作数、有符号的算术操作数、数据加载 (Load) / 数据保存 (Store) 指令的数据地址字节偏移量和分支指令中相对程序计数器 (PC) 的有符号偏移量;

address: 为地址。

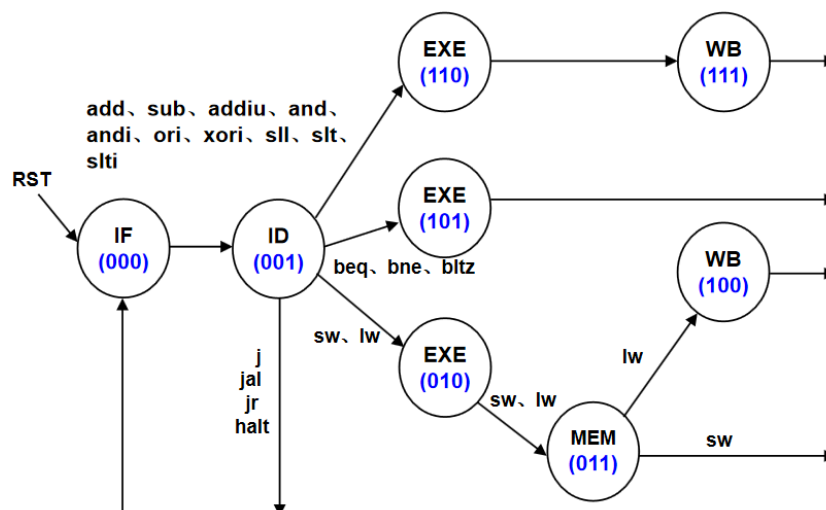


图 2 多周期 CPU 状态转移图

状态的转移有的是无条件的，例如从 IF 状态转移到 ID 就是无条件的；有些是有条件的，例如 EXE 状态之后不止一个状态，到底转向哪个状态由该指令功能，即指令操作码决定。每个状态代表一个时钟周期。

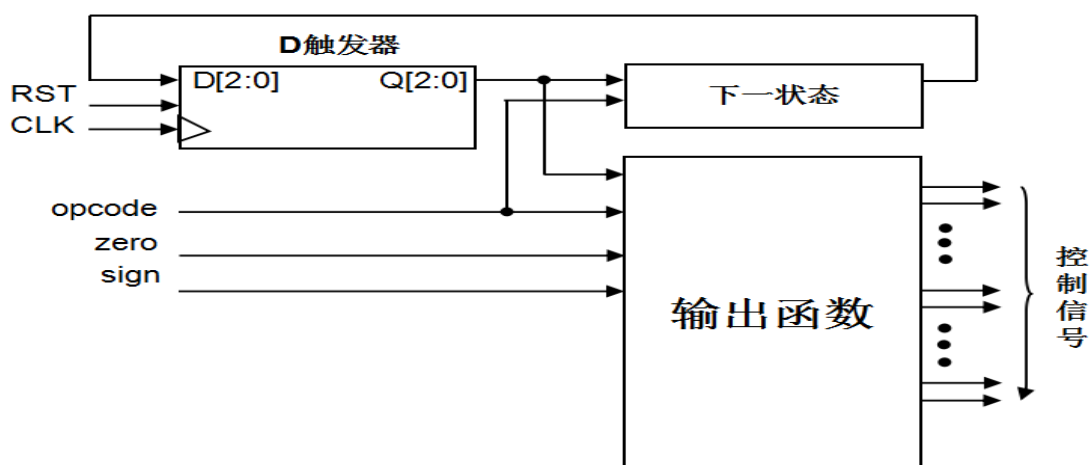


图 3 多周期 CPU 控制部件的原理结构图

图 3 是多周期 CPU 控制部件的电路结构，三个 D 触发器用于保存当前状态，是时序逻辑电路，RST 用于初始化状态“000”，另外两个部分都是组合逻辑电路，一个用于产生下一个阶段的状态，另一个用于产生每个阶段的控制信号。从图上可看出，下个状态取决于指令操作码和当前状态；而每个阶段的控制信号取决于指令操作码、当前状态和反映运算结果的状态 zero 标志和符号 sign 标志。

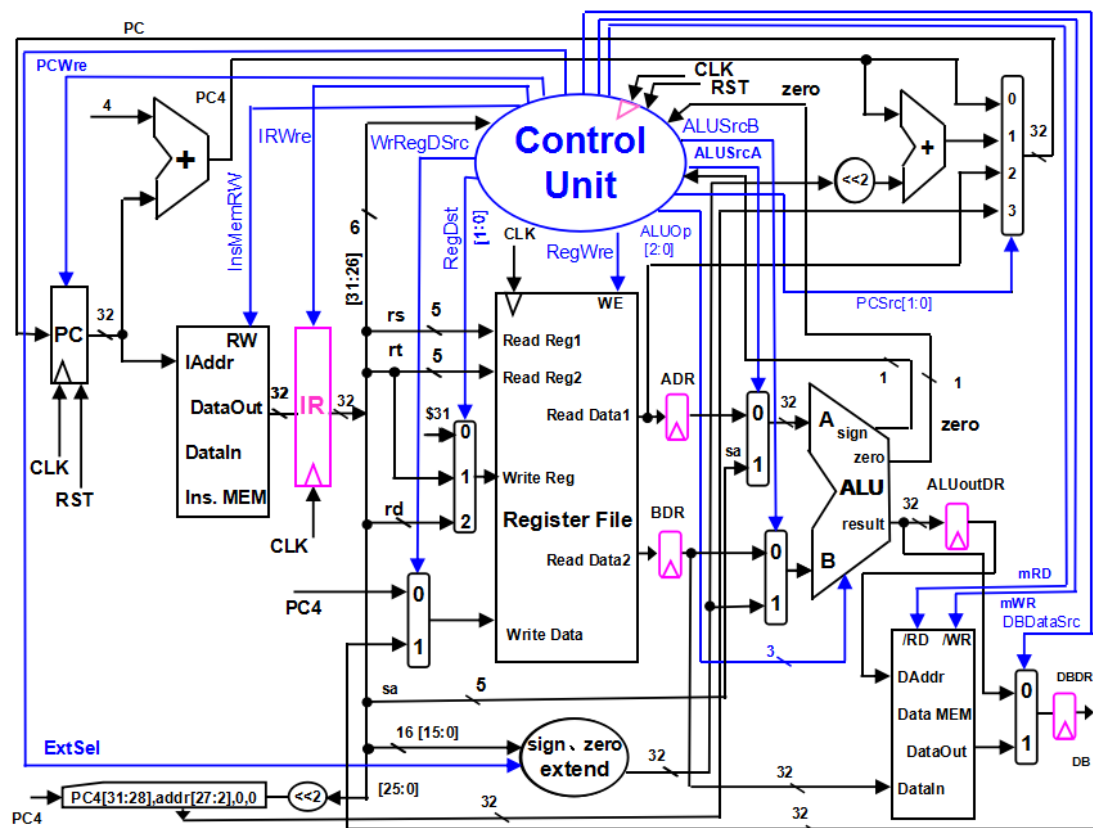


图 4 多周期 CPU 数据通和控制线路图

图 4 是一个简单的基本上能够在多周期 CPU 上完成所要求设计的指令功能的数据通路和必要的控制线路图。其中指令和数据各存储在不同存储器中，即有指令存储器和数据存储器。访问存储器时，先给出内存地址，然后由读或写信号控制操作。对于寄存器组，给出寄存器地址（编号），读操作时不需要时钟信号，输出端就直接输出相应数据；而在写操作时，在 WE 使能信号为 1 时，在时钟边沿触发将数据写入寄存器。图中控制信号功能如表 1 所示，表 2 是 ALU 运算功能表。

特别提示，图上增加 IR 指令寄存器，目的是使指令代码保持稳定，pc 写使能控制信号 PCWre，是确保 pc 适时修改，原因都是和多周期工作的 CPU 有关。ADR、BDR、ALUoutDR、DBDR 四个寄存器不需要写使能信号，其作用是切分数据通路，将大组合逻辑切分为若干个小组合逻辑，大延迟变为多个分段小延迟。

表 1 控制信号作用

控制信号名	状态“0”	状态“1”
RST	对于 PC，初始化 PC 为程序首地址	对于 PC，PC 接收下一条指令地址
PCWre	PC 不更改，相关指令：halt，另外，除‘000’状态之外，其余状态慎改 PC 的值。	PC 更改，相关指令：除指令 halt 外，另外，在‘000’状态时，修改 PC 的值合适。
ALUSrcA	来自寄存器堆 data1 输出，相关指令：add、sub、addiu、and、andi、ori、xori、slt、slti、sw、lw、beq、bne、bltz	来自移位数 sa，同时，进行(zero-extend)sa，即 $\{27\{1'b0\},sa\}$ ，相关指令：sll

ALUSrcB	来自寄存器堆 data2 输出，相关指令：add、sub、and、slt、sll、beq、bne、bltz	来自 sign 或 zero 扩展的立即数，相关指令：addiu、andi、ori、xori、slti、lw、sw
DBDataSrc	来自 ALU 运算结果的输出，相关指令：add、sub、addiu、and、andi、ori、xori、sll、slt、slti	来自数据存储器（Data MEM）的输出，相关指令：lw
RegWre	无写寄存器组寄存器，相关指令：beq、bne、bltz、j、sw、jr、halt	寄存器组寄存器写使能，相关指令：add、sub、addiu、and、andi、ori、xori、sll、slt、slti、lw、jal
WrRegDSrc	写入寄存器组寄存器的数据来自 pc+4(pc4)，相关指令：jal，写 \$31	写入寄存器组寄存器的数据来自 ALU 运算结果或存储器读出的数据，相关指令：add、addiu、sub、and、andi、ori、xori、sll、slt、slti、lw
InsMemRW	写指令存储器	读指令存储器(Ins. Data)
mRD	存储器输出高阻态	读数据存储器，相关指令：lw
mWR	无操作	写数据存储器，相关指令：sw
IRWre	IR(指令寄存器)不更改	IR 寄存器写使能。向指令存储器发出读指令代码后，这个信号也接着发出，在时钟上升沿，IR 接收从指令存储器送来的指令代码。与每条指令都相关。
ExtSel	(zero-extend) immediate ，相关指令：andi、xori、ori；	(sign-extend) immediate ，相关指令：addiu、slti、lw、sw、beq、bne、bltz；
PCSrc[1..0]	00: $pc \leftarrow pc+4$ ，相关指令：add、addiu、sub、and、andi、ori、xori、slt、slti、sll、sw、lw、beq(zero=0)、bne(zero=1)、bltz(sign=0)； 01: $pc \leftarrow pc+4+(sign-extend)immediate \times 4$ ，相关指令：beq(zero=1)、bne(zero=0)、bltz(sign=1)； 10: $pc \leftarrow rs$ ，相关指令：jr； 11: $pc \leftarrow \{pc[31:28], addr[27:2], 2'b00\}$ ，相关指令：j、jal；	
RegDst[1..0]	写寄存器组寄存器的地址，来自： 00: 0x1F(\$31)，相关指令：jal，用于保存返回地址 ($\$31 \leftarrow pc+4$)； 01: rt 字段，相关指令：addiu、andi、ori、xori、slti、lw； 10: rd 字段，相关指令：add、sub、and、slt、sll； 11: 未用；	
ALUOp[2..0]	ALU 8 种运算功能选择(000-111)，看功能表	

相关部件及引脚说明：

Instruction Memory: 指令存储器

Iaddr, 指令地址输入端口

DataIn, 存储器数据输入端口

DataOut, 存储器数据输出端口

RW, 指令存储器读写控制信号, 为 0 写, 为 1 读

Data Memory: 数据存储器

Daddr, 数据地址输入端口

DataIn, 存储器数据输入端口

DataOut, 存储器数据输出端口

/RD, 数据存储器读控制信号, 为 0 读

/WR, 数据存储器写控制信号, 为 0 写

Register File: 寄存器组

Read Reg1, rs 寄存器地址输入端口

Read Reg2, rt 寄存器地址输入端口

Write Reg, 将数据写入的寄存器, 其地址输入端口 (rt、rd)

Write Data, 写入寄存器的数据输入端口

Read Data1, rs 寄存器数据输出端口

Read Data2, rt 寄存器数据输出端口

WE, 写使能信号, 为 1 时, 在时钟边沿触发写入

IR: 指令寄存器, 用于存放正在执行的指令代码

ALU: 算术逻辑单元

result, ALU 运算结果

zero, 运算结果标志, 结果为 0, 则 zero=1; 否则 zero=0

sign, 运算结果标志, 结果最高位为 0, 则 sign=0, 正数; 否则, sign=1, 负数

表 2 ALU 运算功能表

ALUOp[2..0]	功能	描述
000	$Y = A + B$	加
001	$Y = A - B$	减
010	$Y = B \ll A$	B 左移 A 位
011	$Y = A \vee B$	或
100	$Y = A \wedge B$	与
101	$Y = (A < B) ? 1 : 0$	比较 A<B 不带符号
110	$Y = (((A < B) \& \& (A[31] == B[31])) \vee (((A[31] == 1 \& \& B[31] == 0))) ? 1 : 0$	比较 A<B 带符号
111	$Y = A \oplus B$	异或

四. 实验器材

电脑一台, Xilinx Vivado 软件一套, Basys3板一块。

五. 实验过程与结果

CPU设计:

1. ALU (算术逻辑运算单元)

```
module CPU2_ALU(  
    input wire [31:0]data1,  
    input wire [31:0]data2,  
    input wire [2:0]ALUOp,  
    output reg [31:0]result,  
    output reg sign,  
    output reg zero  
);  
always@(data1 or data2 or ALUOp) begin  
    case(ALUOp)  
        3'b000: result <= data1 + data2;  
        3'b001: result <= data1 - data2;  
        3'b010: result <= data2<<data1;  
        3'b011: result <= data1 | data2;  
        3'b100: result <= data1 & data2;  
        3'b101: result <=(data1<data2) ? 1:0;  
        3'b110: result <=((data1<data2) && (data1[31] == data2[31])) ||(( data1[31] ==1 && data2[31] == 0))) ? 1:0;  
        3'b111: result <= data1^data2;  
    endcase  
end  
always@(*)  
begin  
    if(result==0) zero<=1;  
    else zero<=0;  
    if(result[31]==0) sign<=0;  
    else sign<=1;  
end  
endmodule
```

模块输入：两个操作数 data1,data2 和 ALU 控制信号 ALUOp。

模块输出：运算结果 result，符号标志 sign 和零标志 zero。

模块原理：当 data1, data2 或 ALUOp 改变时，ALU 进行运算。根据 ALUOp 的值 0-7，ALU 对两个操作数分别进行加法、减法、移位、按位或、按位与、无符号比较、有符号比较、异或八种运算，并输出运算结果。若结果为 0，零标志=1，否则零标志=0；若结果为正，符号标志=0，否则=1。

2. ALUA, ALUB (两个 ALU 输入选择器)

```

module CPU2_ALUA(
    input wire [31:0]Read_ADR,
    input wire [4:0]sa,
    input wire ALUSrcA,
    output wire [31:0]data1
);
    wire [31:0]shift;
    assign shift = {27'b0000_0000_0000_0000_0000_0000,sa};
    assign data1=ALUSrcA?shift:Read_ADR;
endmodule

module CPU2_ALUB(
    input wire [31:0]Read_BDR,
    input wire ALUSrcB,
    input wire [15:0]immediate,
    input wire ExtSel,
    output wire [31:0]data2
);
    reg [31:0]Extend_Result;
    always@(*)
    begin
        if(ExtSel==0) Extend_Result<={{16{1'b0}},immediate};
        else
            if(immediate[15]==0) Extend_Result<={{16{1'b0}},immediate}
            else begin
                Extend_Result[31:16]<=16'b1111_1111_1111_1111;
                Extend_Result[15:0]<=immediate;
            end
        end
    assign data2=ALUSrcB?Extend_Result:Read_BDR;
endmodule

```

ALUA 是 ALU 输入 data1 的选择器。

模块输入：寄存器读出结果 Read_ADR, 移位数 sa, ALU 输入控制信号 ALUSrcA。

模块输出：ALU 的输入 data1。

模块原理：当 ALUSrcA=0, 输出寄存器读出结果。当 ALUSrcA=1, 输出扩展到 32 位的移位数。

ALUB 是 ALU 输入 data2 的选择器。

模块输入：寄存器读出结果 Read_BDR, 立即数 immediate, ALU 输入控制信号 ALUSrcB, 扩展控制信号 ExtSel。

模块输出：ALU 的输入 data2。

模块原理：当 ALUSrc=0, 输出寄存器读出结果。当 ALUSrc=1, 输出符号扩展(ExtSel=1)或位扩展(ExtSel=0)的 32 位立即数。

3. ALUR (ALU 输出寄存器), Data1R 和 Data2R (寄存器读出结果寄存器), DataR (数据存储寄存器读出结果寄存器)

由于四个寄存器的结构相同，故只放出 ALUR 的代码。

```
module CPU2_ALUR(  
    input CLK,  
    input [31:0]result,  
    output reg[31:0]result_out  
);  
  
    initial begin  
        result_out<=0;  
    end  
  
    always@(negedge CLK)  
    begin  
        result_out<=result;  
    end  
endmodule
```

ALUR 的输入有：CLK, ALU 运算结果。

ALUR 的输出为：ALU 结果寄存器的输出 result_out。

模块原理：在时钟下降沿，ALU 结果更新。其余三个寄存器同理。

4. 控制单元 (Control Unit)

```
module CPU2_ControlUnit(  
    input CLK,  
    input Reset,  
    input sign,  
    input zero,  
    input [5:0]op,  
    input [5:0]sig,  
    input [10:0]func,  
    output reg RegWre,  
    output reg [2:0]ALUOp,  
    output reg ALUSrcA,  
    output reg ALUSrcB,  
    output reg mRD,  
    output reg mWR,  
    output reg DBDataSrc,  
    output reg PCWre,  
    output reg ExtSel,  
    output reg InsMemRW,  
    output reg IRWre,  
    output reg WrRegDSrc,  
    output reg [1:0]RegDst,  
    output reg [1:0]PCSrc,  
    output reg [2:0]cur_state,  
    output reg [2:0]nxt_state  
);
```

```

50 ○ always@(negedge CLK)
51 ○ begin
52 ○     if(Reset==0) cur_state=3'b000;
53 ○     else cur_state=nxt_state;
54 ○     begin
55 ○         case(cur_state)
56 ○             3'b000:begin
57 ○                 nxt_state=3'b001;
58 ○                 end
59 ○             3'b001:begin
60 ○                 if(op==6'b000101 || op==6'b000100 || op==6'b000001)
61 ○                     nxt_state=3'b101;
62 ○                 else if(op==6'b101011 || op==6'b100011)
63 ○                     nxt_state=3'b010;
64 ○                 else if(op==6'b000010 || op==6'b000011 || op==6'b111111 || (op==6'b000000 && sig==6'b001000))
65 ○                     nxt_state=3'b000;
66 ○                 else
67 ○                     nxt_state=3'b110;
68 ○                 end
69 ○             3'b010:begin
70 ○                 nxt_state=3'b011;
71 ○                 end
72 ○             3'b011:begin
73 ○                 if(op==6'b100011)
74 ○                     nxt_state=3'b100;
75 ○                 else
76 ○                     nxt_state=3'b000;
77 ○                 end
78 ○             3'b100:begin
79 ○                 nxt_state=3'b000;
80 ○                 end
81 ○             3'b101:begin
82 ○                 nxt_state=3'b000;
83 ○                 end
84 ○             3'b110:begin
85 ○                 nxt_state=3'b111;
86 ○                 end
87 ○             3'b111:begin
88 ○                 nxt_state=3'b000;
89 ○                 end
90 ○         endcase
91 ○     end
92 ○ end
93
94 ○ always@(*)
95 ○ begin
96 ○     InsMemRW=1;
97 ○     case(op)
98 ○         6'b000000:
99 ○             case(func)
100 ○                 11'b00000_100000: begin//add
101 ○                     if(cur_state==7)begin
102 ○                         PCWre<=1;
103 ○                         RegWre<=0;

```

104	○	IRWre<=1;
105	○	end
106	○	if (cur_state==0) begin
107	○	PCWre<=0;
108	○	end
109	○	if (cur_state==1) begin
110	○	PCWre<=0;
111	○	ALUSrcA<=0;
112	○	ALUSrcB<=0;
113	○	DBDataSrc<=0;
114	○	RegWre<=0;
115	○	IRWre<=0;
116	○	PCSrc<=2'b00;
117	○	RegDst<=2'b10;
118	○	ALUOp<=3'b000;
119	○	end
120	○	if (cur_state==6) begin
121	○	RegWre<=1;
122	○	WrRegDSrc<=1;
123	○	end
124	○	end
125	○	11'b00000_100010: begin//sub
126	○	if (cur_state==7) begin
127	○	PCWre<=1;
128	○	RegWre<=0;
129	○	IRWre<=1;
130	○	end
131	○	if (cur_state==0) begin
132	○	PCWre<=0;
133	○	end
134	○	if (cur_state==1) begin
135	○	PCWre<=0;
136	○	ALUSrcA<=0;
137	○	ALUSrcB<=0;
138	○	DBDataSrc<=0;
139	○	RegWre<=0;
140	○	IRWre<=0;
141	○	PCSrc<=2'b00;
142	○	RegDst<=2'b10;
143	○	ALUOp<=3'b001;
144	○	end
145	○	if (cur_state==6) begin
146	○	RegWre<=1;
147	○	WrRegDSrc<=1;
148	○	end
149	○	end
150	○	11'b00000_100100: begin//and
151	○	if (cur_state==7) begin
152	○	PCWre<=1;
153	○	RegWre<=0;
154	○	IRWre<=1;
155	○	end
156	○	if (cur_state==0) begin
157	○	PCWre<=0;

158	⬇		end
159	⬇	○	if(cur_state==1)begin
160	⬇	○	PCWre<=0;
161	⬇	○	ALUSrcA<=0;
162	⬇	○	ALUSrcB<=0;
163	⬇	○	DBDataSrc<=0;
164	⬇	○	RegWre<=0;
165	⬇	○	IRWre<=0;
166	⬇	○	PCSrc<=2'b00;
167	⬇	○	RegDst<=2'b10;
168	⬇	○	ALUOp<=3'b100;
169	⬇	⬇	end
170	⬇	○	if(cur_state==6)begin
171	⬇	○	RegWre<=1;
172	⬇	○	WrRegDSrc<=1;
173	⬇	⬇	end
174	⬇	⬇	end
175	⬇		11'b00000_100101: begin//or
176	⬇	○	if(cur_state==7)begin
177	⬇	○	PCWre<=1;
178	⬇	○	RegWre<=0;
179	⬇	○	IRWre<=1;
180	⬇	⬇	end
181	⬇	○	if(cur_state==0)begin
182	⬇	○	PCWre<=0;
183	⬇	⬇	end
184	⬇	○	if(cur_state==1)begin
185	⬇	○	PCWre<=0;
186	⬇	○	ALUSrcA<=0;
187	⬇	○	ALUSrcB<=0;
188	⬇	○	DBDataSrc<=0;
189	⬇	○	RegWre<=0;
190	⬇	○	IRWre<=0;
191	⬇	○	PCSrc<=2'b00;
192	⬇	○	RegDst<=2'b10;
193	⬇	○	ALUOp<=3'b011;
194	⬇	⬇	end
195	⬇	○	if(cur_state==6)begin
196	⬇	○	RegWre<=1;
197	⬇	○	WrRegDSrc<=1;
198	⬇	⬇	end
199	⬇	⬇	end
200	⬇		11'b00000_101010:begin//slt
201	⬇	○	if(cur_state==7)begin
202	⬇	○	PCWre<=1;
203	⬇	○	RegWre<=0;
204	⬇	○	IRWre<=1;
205	⬇	⬇	end
206	⬇	○	if(cur_state==0)begin
207	⬇	○	PCWre<=0;
208	⬇	⬇	end
209	⬇	○	if(cur_state==1)begin
210	⬇	○	PCWre<=0;
211	⬇	○	ALUSrcA<=0;

```

212 ○ ALUSrcB<=0;
213 ○ DBDataSrc<=0;
214 ○ RegWre<=0;
215 ○ IRWre<=0;
216 ○ PCSrc<=2'b00;
217 ○ RegDst<=2'b10;
218 ○ ALUOp<=3'b101;//no sign
219 ○ end
220 ○ if(cur_state==6)begin
221 ○ RegWre<=1;
222 ○ WrRegDSrc<=1;
223 ○ end
224 ○ end
225 ○ 11'b00000_001000:begin//jr
226 ○ if(cur_state==1)begin
227 ○ PCWre<=1;
228 ○ RegWre<=0;
229 ○ IRWre<=1;
230 ○ end
231 ○ if(cur_state==0)begin
232 ○ PCWre<=0;
233 ○ PCSrc<=2'b10;
234 ○ end
235 ○ end
236 ○ default:begin//sll
237 ○ if(cur_state==7)begin
238 ○ PCWre<=1;

```

```

239 ○ RegWre<=0;
240 ○ IRWre<=1;
241 ○ end
242 ○ if(cur_state==0)begin
243 ○ PCWre<=0;
244 ○ end
245 ○ if(cur_state==1)begin
246 ○ PCWre<=0;
247 ○ ALUSrcA<=1;
248 ○ ALUSrcB<=0;
249 ○ DBDataSrc<=0;
250 ○ RegWre<=0;
251 ○ IRWre<=0;
252 ○ PCSrc<=2'b00;
253 ○ RegDst<=2'b10;
254 ○ ALUOp<=3'b010;
255 ○ end
256 ○ if(cur_state==6)begin
257 ○ RegWre<=1;
258 ○ WrRegDSrc<=1;
259 ○ ALUSrcA<=0;
260 ○ end
261 ○ end
262 ○ endcase
263 ○ 6'b001001:begin//addiu;
264 ○ if(cur_state==7)begin
265 ○ PCWre<=1;

```


266	○	RegWre<=0;
267	○	IRWre<=1;
268	⊖	end
269	⊖	if(cur_state==0)begin
270	○	PCWre<=0;
271	⊖	end
272	⊖	if(cur_state==1)begin
273	○	PCWre<=0;
274	○	ALUSrcA<=0;
275	○	ALUSrcB<=1;
276	○	DBDataSrc<=0;
277	○	RegWre<=0;
278	○	IRWre<=0;
279	○	ExtSel<=1;
280	○	RegDst<=2' b01;
281	○	PCSrc<=2' b00;
282	○	ALUOp<=3' b000;
283	⊖	end
284	⊖	if(cur_state==6)begin
285	○	RegWre<=1;
286	○	WrRegDSrc<=1;
287	⊖	end
288	⊖	end
289	⊖	6'b001100:begin//andi
290	⊖	if(cur_state==7)begin
291	○	PCWre<=1;
292	○	RegWre<=0;
293	○	IRWre<=1;
294	⊖	end
295	⊖	if(cur_state==0)begin
296	○	PCWre<=0;
297	⊖	end
298	⊖	if(cur_state==1)begin
299	○	PCWre<=0;
300	○	ALUSrcA<=0;
301	○	ALUSrcB<=1;
302	○	DBDataSrc<=0;
303	○	RegWre<=0;
304	○	IRWre<=0;
305	○	ExtSel<=0;
306	○	RegDst<=2' b01;
307	○	PCSrc<=2' b00;
308	○	ALUOp<=3' b100;
309	⊖	end
310	⊖	if(cur_state==6)begin
311	○	RegWre<=1;
312	○	WrRegDSrc<=1;
313	⊖	end
314	⊖	end
315	⊖	6'b001101:begin//ori
316	⊖	if(cur_state==7)begin
317	○	PCWre<=1;
318	○	RegWre<=0;
319	○	IRWre<=1;

```

320 ○
321 ○ ○ if(cur_state==0)begin
322 ○ ○   PCWre<=0;
323 ○ ○   end
324 ○ ○ if(cur_state==1)begin
325 ○ ○   PCWre<=0;
326 ○ ○   ALUSrcA<=0;
327 ○ ○   ALUSrcB<=1;
328 ○ ○   DBDataSrc<=0;
329 ○ ○   RegWre<=0;
330 ○ ○   IRWre<=0;
331 ○ ○   ExtSel<=0;
332 ○ ○   RegDst<=2' b01;
333 ○ ○   PCSrc<=2' b00;
334 ○ ○   ALUOp<=3' b011;
335 ○ ○   end
336 ○ ○ if(cur_state==6)begin
337 ○ ○   RegWre<=1;
338 ○ ○   WrRegDSrc<=1;
339 ○ ○   end
340 ○ ○ end
341 ○ 6' b001010:begin//slti
342 ○ ○ if(cur_state==7)begin
343 ○ ○   PCWre<=1;
344 ○ ○   RegWre<=0;
345 ○ ○   IRWre<=1;
346 ○ ○   end

347 ○ ○ if(cur_state==0)begin
348 ○ ○   PCWre<=0;
349 ○ ○   end
350 ○ ○ if(cur_state==1)begin
351 ○ ○   PCWre<=0;
352 ○ ○   ALUSrcA<=0;
353 ○ ○   ALUSrcB<=1;
354 ○ ○   DBDataSrc<=0;
355 ○ ○   RegWre<=0;
356 ○ ○   IRWre<=0;
357 ○ ○   ExtSel<=1;
358 ○ ○   RegDst<=2' b01;
359 ○ ○   PCSrc<=2' b00;
360 ○ ○   ALUOp<=3' b110;
361 ○ ○   end
362 ○ ○ if(cur_state==6)begin
363 ○ ○   RegWre<=1;
364 ○ ○   WrRegDSrc<=1;
365 ○ ○   end
366 ○ ○ end
367 ○ 6' b101011:begin//sw
368 ○ ○ if(cur_state==3)begin
369 ○ ○   PCWre<=1;
370 ○ ○   RegWre<=0;
371 ○ ○   IRWre<=1;
372 ○ ○   end
373 ○ ○ if(cur_state==0)begin

```

```

374 ○ PCWre<=0;
375 ○ end
376 ○ if(cur_state==1)begin
377 ○ PCWre<=0;
378 ○ ALUSrcA<=0;
379 ○ ALUSrcB<=1;
380 ○ DBDataSrc<=0; //没提到这个 不过应该无所谓
381 ○ RegWre<=0;
382 ○ IRWre<=0;
383 ○ mWR<=1;
384 ○ ExtSel<=1;
385 ○ PCSrc<=2'b00;
386 ○ ALUOp<=3'b000; //sign
387 ○ end
388 ○ if(cur_state==2)begin
389 ○ mWR<=1;
390 ○ end
391 ○ end
392 ○ 6'b100011:begin//lw
393 ○ if(cur_state==4)begin
394 ○ PCWre<=1;
395 ○ RegWre<=0;
396 ○ IRWre<=1;
397 ○ end
398 ○ if(cur_state==0)begin
399 ○ PCWre<=0;
400 ○ end

```

```

401 ○ if(cur_state==1)begin
402 ○ PCWre<=0;
403 ○ ALUSrcA<=0;
404 ○ ALUSrcB<=1;
405 ○ DBDataSrc<=1;
406 ○ RegWre<=0;
407 ○ IRWre<=0;
408 ○ ExtSel<=1;
409 ○ PCSrc<=2'b00;
410 ○ ALUOp<=3'b000; //sign
411 ○ RegDst<=2'b01;
412 ○ end
413 ○ if(cur_state==2)begin
414 ○ mRD<=1;
415 ○ end
416 ○ if(cur_state==3)begin
417 ○ RegWre<=1;
418 ○ WrRegDSrc<=1;
419 ○ end
420 ○ end
421 ○ 6'b000100:begin//beq
422 ○ if(cur_state==5)begin
423 ○ PCWre<=1;
424 ○ RegWre<=0;
425 ○ IRWre<=1;
426 ○ end
427 ○ if(cur_state==0)begin

```

```

428 ○          PCWre<=0;
429 ○          end
430 ○          if(cur_state==1)begin
431 ○              PCWre<=0;
432 ○              ALUSrcA<=0;
433 ○              ALUSrcB<=0;
434 ○              DBDataSrc<=0;
435 ○              RegWre<=0;
436 ○              IRWre<=0;
437 ○              ExtSel<=1;
438 ○              if(zero==1)PCSrc<=2'b01;
439 ○              else PCSrc<=2'b00;
440 ○              ALUOp<=3'b001;
441 ○          end
442 ○          end
443 ○          6'b000101:begin//bne
444 ○              if(cur_state==5)begin
445 ○                  PCWre<=1;
446 ○                  RegWre<=0;
447 ○                  IRWre<=1;
448 ○              end
449 ○              if(cur_state==0)begin
450 ○                  PCWre<=0;
451 ○                  end
452 ○              if(cur_state==1)begin
453 ○                  PCWre<=0;
454 ○                  ALUSrcA<=0;
455 ○                  ALUSrcB<=0;
456 ○                  DBDataSrc<=0;
457 ○                  RegWre<=0;
458 ○                  IRWre<=0;
459 ○                  ExtSel<=1;
460 ○                  if(zero==1)PCSrc<=2'b00;
461 ○                  else PCSrc<=2'b01;
462 ○                  ALUOp<=3'b001;
463 ○              end
464 ○              end
465 ○              6'b000001:begin//bltz
466 ○                  if(cur_state==5)begin
467 ○                      PCWre<=1;
468 ○                      RegWre<=0;
469 ○                      IRWre<=1;
470 ○                  end
471 ○                  if(cur_state==0)begin
472 ○                      PCWre<=0;
473 ○                  end
474 ○                  if(cur_state==1)begin
475 ○                      PCWre<=0;
476 ○                      ALUSrcA<=0;
477 ○                      ALUSrcB<=0;
478 ○                      DBDataSrc<=0;
479 ○                      RegWre<=0;
480 ○                      IRWre<=0;
481 ○                      ExtSel<=1;

```

```

482 ○ if(sign==0)PCSrc<=2'b00;
483 ○ else PCSrc<=2'b01;
484 ○ end
485 ○ end
486 ○ 6'b000010:begin//j
487 ○ if(cur_state==1)begin
488 ○ PCWre<=1;
489 ○ RegWre<=0;
490 ○ IRWre<=1;
491 ○ end
492 ○ if(cur_state==0)begin
493 ○ PCWre<=0;
494 ○ RegWre<=0;
495 ○ PCSrc<=2'b11;
496 ○ end
497 ○ end
498 ○ 6'b000011:begin//jal
499 ○ if(cur_state==1)begin
500 ○ PCWre<=1;
501 ○ RegWre<=0;
502 ○ IRWre<=1;
503 ○ end
504 ○ if(cur_state==0)begin
505 ○ PCWre<=0;
506 ○ RegWre<=1;
507 ○ WrRegDSrc<=0;
508 ○ PCSrc<=2'b11;
509 ○ end

510 ○ end
511 ○ 6'b111111:begin//halt
512 ○ if(cur_state==1)begin
513 ○ PCWre<=0;
514 ○ RegWre<=0;
515 ○ IRWre<=1;
516 ○ PCSrc<=2'b11;
517 ○ end
518 ○ if(cur_state==0)begin
519 ○ PCWre<=0;
520 ○ RegWre<=0;
521 ○ IRWre<=0;
522 ○ PCSrc<=2'b11;
523 ○ end
524 ○ end
525 ○ 6'b001110:begin//xori
526 ○ if(cur_state==7)begin
527 ○ PCWre<=1;
528 ○ RegWre<=0;
529 ○ IRWre<=1;
530 ○ end
531 ○ if(cur_state==0)begin
532 ○ PCWre<=0;
533 ○ end
534 ○ if(cur_state==1)begin
535 ○ PCWre<=0;
536 ○ ALUSrcA<=0;

```

```

537 | ○ | ALUSrcB<=1;
538 | ○ | DBDataSrc<=0;
539 | ○ | RegWre<=0;
540 | ○ | IRWre<=0;
541 | ○ | ExtSel<=0;
542 | ○ | RegDst<=2'b01;
543 | ○ | PCSrc<=2'b00;
544 | ○ | ALUOp<=3'b111;
545 | ○ | end
546 | ○ | if(cur_state==6)begin
547 | ○ |     RegWre<=1;
548 | ○ |     WrRegDSrc<=1;
549 | ○ | end
550 | ○ | end
551 | ○ | endcase
552 | ○ | end
553 | ○ | endmodule
554 |

```

模块输入：CLK, Reset, 指令的 op 字段, 指令的 func 字段, 零信号, 符号信号, 指令的最后六位 sig。

模块输出：各个控制信号。

模块原理：控制单元的第一部分是一个 D 触发器, 在时钟下降沿时, 根据当前的指令, 更新状态。

控制单元的第二部分是一个嵌套的 case 语句, 根据当前的指令和状态产生对应的控制信号。

控制信号表如下：

Add, sub, addiu, and

多周期CPU控制信号表											
指令/控制信号		Reset	ALUSrcA	ALUSrcB	PCWre	IRWre	InsMemRW	DBDataSrc	RegWre	WrRegDSrc	mRD
add	IF	1			1	1	1		0		
	ID	1			0		1				
	EXE	1	0	0	0	0	1	0	0		
	WB	1			0		1		1	1	
sub	IF	1			1	1	1		0		
	ID	1			0		1				
	EXE	1	0	0	0	0	1	0	0		
	WB	1			0		1		1	1	
addiu	IF	1			1	1	1		0		
	ID	1			0		1				
	EXE	1	0	1	0	0	1	0	0		
	WB	1			0		1		1	1	
and	IF	1			1	1	1		0		
	ID	1			0		1				
	EXE	1	0	0	0	0	1	0	0		
	WB	1			0		1		1	1	
	IF	1			1	1	1		0		

多周期CPU控制信号表											
PCWre	IRWre	InsMemRW	DBDataSrc	RegWre	WrRegDSrc	mRD	mRW	ExtSel	RegDst	PCSrc	ALUOp
1	1	1		0							
0		1									
0	0	1	0	0					10	00	000
0		1		1	1						
1	1	1		0							
0		1									
0	0	1	0	0					10	00	001
0		1		1	1						
1	1	1		0							
0		1									
0	0	1	0	0				1	01	00	000
0		1		1	1						
1	1	1		0							
0		1									
0	0	1	0	0					10	00	100
0		1		1	1						
1	1	1		0							

Andi, or, ori, xori, sll

andi	IF	1			1	1	1		0			
	ID	1			0		1					
	EXE	1	0	1	0	0	1	0	0			
or	WB	1			0		1		1	1		
	IF	1			1	1	1		0			
	ID	1			0		1					
ori	EXE	1	0	0	0	0	1	0	0			
	WB	1			0		1		1	1		
	IF	1			1	1	1		0			
xori	ID	1			0		1					
	EXE	1	0	1	0	0	1	0	0			
	WB	1			0		1		1	1		
sll	IF	1			1	1	1		0			
	ID	1			0		1					
	EXE	1	1	0	0	0	1	0	0			
	WB	1	0		0		1		1	1		

19	1	1	1		0							
20	0		1									
21	0	0		0	0				0	01	00	100
22	0		1		1	1						
23	1	1	1		0							
24	0		1									
25	0	0	1	0	0					10	00	011
26	0		1		1	1						
27	1	1	1		0							
28	0		1									
29	0	0	1	0	0				0	01	00	011
30	0		1		1	1						
31	1	1	1		0							
32	0		1									
33	0	0	1	0	0				0	01	00	111
34	0		1		1	1						
35	1	1	1		0							
36	0		1									
37	0	0	1	0	0					10	00	010
38	0		1		1	1						

Slt, slti, sw, lw

39		vvv	1	0		0	1	1	1	1		
40	slt	IF	1			1	1	1	0			
41		ID	1			0		1				
42		EXE	1	0	0	0		1	0	0		
43	slti	WB	1			0		1		1	1	
44		IF	1			1	1	1		0		
45		ID	1			0		1				
46	sw	EXE	1	0	1	0	0	1	0	0		
47		WB	1			0		1		1	1	
48		IF	1			1	1	1		0		
49	lw	ID	1			0		1				
50		EXE	1	0	1	0	0	1	0	0		
51		MEM	1			0		1				
52		IF	1			1	1	1		0		
53		ID	1			0		1				
54		EXE	1	0	1	0	0	1	1	0		
55		MEM	1			0		1			1	1
56		WB	1			0		1		1		
		IF	1			1	1	1		0		

0	0	1	0	0	1				10	00	010
1	1	1		0							
0		1									
0		1	0	0					10	00	110
0		1		1	1						
1	1	1		0							
0		1									
0	0	1	0	0				1	01	00	110
0		1		1	1						
1	1	1		0							
0		1									
0	0	1	0	0				1		00	000
0		1					1				
1	1	1		0							
0		1	1	0					01	00	000
0		1			1	1					
0		1		1	1				01		
1	1	1		0							

Beq, bne, bltz, jal, jr, j, halt.

56		IF	1			1	1	1		0		
57	beq	ID	1			0		1				
58		EXE	1	0	0	0	0	1	0	0		
59		IF	1			1	1	1		0		
60	bne	ID	1			0		1				
61		EXE	1	0	0	0	0	1	0	0		
62		IF	1			1	1	1		0		
63	bltz	ID	1			0		1				
64		EXE	1	0	0	0	0	1	0	0		
65		IF	1			1	1	1		0		
66	jal	ID	1			0		1		1	0	
67		IF	1			1	1	1		0		
68	jr	ID	1			0		1				
69		IF	1			1	1	1		0		
70	j	ID	1			0		1		0		
71		IF	1			0	1	1		0		
72	halt	ID	1			0	0	1		0		

55	0		1		1	1				01		
56	1	1	1		0							
57	0		1									
58	0	0	1	0	0				1	zero=0:00 zero=1	001	
59	1	1	1		0							
60	0		1									
61	0	0	1	0	0				1	zero=0:01 zero=1	001	
62	1	1	1		0							
63	0		1									
64	0	0	1	0	0				1	sign=0: 00 sign=1	001	
65	1	1	1		0							
66	0		1		1	0					11	
67	1	1	1		0							
68	0		1								10	
69	1	1	1		0							
70	0		1		0						11	
71	0	1	1		0						11	
72	0	0	1		0						11	

5. 数据存储器（DataMem）


```

module CPU2_DataMem(
    input wire [31:0]Daddr,
    input wire [31:0]DataIn,
    input wire mRD,
    input wire mWR,
    input CLK,
    output wire[31:0]Datamem
);
    reg [7:0]DataMem[0:100];
    integer i;
    initial begin
        for(i=0;i<100;i=i+1)begin
            DataMem[i] = 8'b0000_0000;
        end
    end
    always@(negedge CLK)
    begin
        if(mWR==0) ;//write
        else
            begin
                DataMem[Daddr+3]=DataIn[7:0];
                DataMem[Daddr+2]=DataIn[15:8];
                DataMem[Daddr+1]=DataIn[23:16];
                DataMem[Daddr]=DataIn[31:24];
            end
    end

    assign Datamem[7:0]= mRD?DataMem[Daddr+3]:8'bz;
    assign Datamem[15:8]= mRD?DataMem[Daddr+2]:8'bz;
    assign Datamem[23:16]= mRD?DataMem[Daddr+1]:8'bz;
    assign Datamem[31:24]= mRD?DataMem[Daddr]:8'bz;
endmodule

```

模块输入：CLK，存储器访问地址 Daddr，数据存入内容 DataIn，写存储器控制信号 mWR，读存储器控制信号 mRD。

模块输出：Daddr 处的数据。

模块原理：在时钟下降沿，若 mWR=1，写数据存储器；mWR=0，无操作。若 mRD=1，读数据存储器；mRD=0，输出高阻态。

6. 指令存储器

```

module CPU2_InsMem(
    input wire [31:0] IAddr,
    input wire RW,
    output reg [31:0] IDataOut
);
    reg [7:0] InsMem[0:100];
    initial begin
        $readmemb("E:/Vivado/Instructions for CPU2.txt", InsMem, 0, 100);
    end
    assign RW=1;
    always@(*)
    begin
        if(RW==0);
        else
        begin
            IDataOut[7:0]=InsMem[IAddr+3];
            IDataOut[15:8]=InsMem[IAddr+2];
            IDataOut[23:16]=InsMem[IAddr+1];
            IDataOut[31:24]=InsMem[IAddr];
        end
    end
endmodule

```

模块输入：存储器访问地址 IAddr，读指令控制信号 RW。

模块输出：IAddr 处的指令 IDataOut。

模块原理：由于在 CPU 运行过程中不存在写指令的任务，故 RW=1，存储器始终在读指令的模式。

指令存储器存储的指令内容为：

```

00100100 00000001 00000000 00001000
00110100 00000010 00000000 00000010
00111000 01000011 00000000 00001000
00000000 01100001 00100000 00100010
00000000 10000010 00101000 00100100
00000000 00000101 00101000 10000000
00010000 10100001 11111111 11111110
00001100 00000000 00000000 00010100
00000001 10100001 00100000 00101010
00100100 00001110 11111111 11111110
00000001 00001110 01001000 00101010
00101001 00101010 00000000 00000010
00101001 01001011 00000000 00000000
00000001 01101010 01011000 00100000
00010101 01100010 11111111 11111110
00100100 00001100 11111111 11111110
00100101 10001100 00000000 00000001
00000101 10000000 11111111 11111110
00110000 01001100 00000000 00000010
00001000 00000000 00000000 00010111
10101100 00100010 00000000 00000100
10001100 00101101 00000000 00000100
00000000 11111000 00000000 00001000
11111100 00000000 00000000 00000000

```

对应的 mips 汇编指令为：

```

addiu $1,$0,8
ori $2,$0,2
xori $3,$2,8
sub $4,$3,$1
and $5,$4,$2
sll $5,$5,2
beq $5,$1,-2
jal 0x0000050
slt $8,$13,$1
addiu $14,$0,-2
slt $9,$8,$14
slli $10,$9,2
slli $11,$10,0
add $11,$11,$10
bne $11,$2,-2
addiu $12,$0,-2
addiu $12,$12,1
bltz $12,-2
andi $12,$2,2
j 0x000005C
sw $2,4($1)
lw $13,4($1)
jr $31
halt

```

编译器的代码为：

第一部分是两个函数，分别将十进制整数转换为 5 位和 16 位二进制数。

```

1 //computer
2 #include<iostream>
3 #include<fstream>
4 #include<string>
5 #include<bitset>
6 using namespace std;
7
8 string ToBinary5(int a){
9     string b(5,'0');
10    int i=4;
11    while (a>0)
12    {
13        b[i] = a%2 + '0';
14        i = i - 1;
15        a = a / 2;
16    }
17    return b;
18 }
19
20 string ToBinary16(int a){
21     string b(16,'0');
22     int i=15;
23     if(a==0){
24         return b;
25     }
26     else if(a>0){
27         while (a>0)
28         {

```

```

28 {
29     b[i] = a % 2 + '0';
30     i = i - 1;
31     a = a / 2;
32 }
33 return b;
34 }
35 else{
36     int c = a*(-1);
37     while (c>0)
38     {
39         b[i] = c % 2 + '0';
40         i = i - 1;
41         c = c / 2;
42     }
43     for(int k=0; k<=15; k++){
44         if(b[k]=='0'){
45             b[k]='1';
46         }
47         else {
48             b[k]='0';
49         }
50     }
51     int d = 15;
52     while(b[d]!='0'){
53         b[d]='0';
54         d--;
55     }
56 }

```

第二部分是编译函数，将输入的 MIPS 指令字符串转换为对应的二进制机器代码字符串。

```

61 string Compile(string a){
62     int op = a.find_first_of(" ");
63     string b = a.substr(0,op);
64     string c;
65     if(b=="addiu"){
66         a.erase(0,5);
67         int rt = a.find_first_of("$");
68         int y = a.find_first_of(",");
69         string d = a.substr(rt+1,y-rt-1);
70         int rt1 = stol(d);
71         string rt2;
72         rt2 = ToBinary5(rt1);
73         a.erase(rt,3);
74         int rs = a.find_first_of("$");
75         int x = a.find_first_of(",");
76         string e = a.substr(rs+1,x-rs-1);
77         int rs1 = stol(e);
78         string rs2;
79         rs2 = ToBinary5(rs1);
80         a.erase(rs,3);
81         int immediate = a.find_last_of(",");
82         string f = a.substr(immediate+1);
83         int immediate1 = stol(f);
84         string immediate2;
85         immediate2 = ToBinary16(immediate1);
86         c = "001001" + rs2.substr(0,2) + " " + rs2.substr(2,3) + rt2 + " " + immediate2.substr(0,8) + " " + immediate2.substr(8,8);
87         cout<<c<<endl;
88     }
89 }

```

第三部分是主函数，实现对文件的读写。

```

418 int main(){
419     ifstream f;
420     f.open("C:/Users/Administrator/Desktop/mips.txt",ios::in);
421     ofstream a;
422     a.open("C:/Users/Administrator/Desktop/binary.txt",ios::out);
423     string* s = new string[25];
424     int i=0;
425     char line[1021] = {0};
426     while(f.getline(line, sizeof(line))){
427         cout<<line<<endl;
428         s[i] = line;
429         s[i] = Compile(s[i]);
430         a <<s[i]<< endl;
431         i++;
432     }
433 }

```

编译后结果为：

```

00100100 00000001 00000000 00001000
00110100 00000010 00000000 00000010
00111000 01000011 00000000 00001000
00000000 01100001 00100000 00100010
00000000 10000010 00101000 00100100
00000000 0000101 00101000 10000000
00010000 00100101 11111111 11111110
00001100 00000000 00000000 00010100
00000001 10100001 01000000 00101010
00100100 00001110 11111111 11111110
00000001 00001110 01001000 00101010
00101001 00101010 00000000 00000010
00101001 01001011 00000000 00000000
00000001 01101010 01011000 00100000
00010100 01001011 11111111 11111110
00100100 00001100 11111111 11111110
00100101 10001100 00000000 00000001
00000101 10000000 11111111 11111110
00110000 01000001 00000000 00000010
00001000 00000000 00000000 00010111
10101100 00100010 00000000 00000100
10001100 00101101 00000000 00000100
00000011 11100000 00000000 00001000
11111100 00000000 00000000 00000000

```

7. 指令寄存器 IR

```

module CPU2_IR(
    input CLK,
    input IRWre,
    input [31:0]Ins_In,
    output [31:0]Ins_Out
);

    reg [31:0]Ins;

    initial begin
        Ins<=32'b0000_0000_0000_0000_0000_0000_0000_0000;
    end

    always@(posedge CLK)begin
        if(IRWre==0);
        else Ins<=Ins_In;
        end

    assign Ins_Out=Ins;
endmodule

```

模块输入：CLK，指令更新控制信号 IRWre，输入指令 Ins_In。

模块输出：输出指令 Ins_Out。

模块原理：在时钟上升沿，若 IRWre=0，指令寄存器输入新指令。（这里和实验原理部分的不一样，是写代码时的失误，但不影响 CPU 的功能）。

8. PC

```

module CPU2_PC(
    input wire PCWre, //0:PC Unchanged 1:PC change
    input wire[31:0]PC_In, //The result of PC calculation multiple cho
    input wire Reset, //0:PC=0 1:PC=new
    input CLK,
    output [31:0]PC_Out//current PC
);
    reg [31:0]Current_PC; //PC Register
    //    initial begin
    //        Current_PC<=0; //Initial register
    //    end
    always@(posedge CLK or negedge Reset)
    begin
        if(Reset==0) Current_PC<=0;
        else begin
            if(PCWre==0) Current_PC<=Current_PC;
            else Current_PC<=PC_In; //Change current PC
        end
    end
    assign PC_Out =Current_PC;
endmodule

```

模块输入：PC 更新控制信号 PCWre，下一个指令地址 PC_In，CLK, Reset.

模块输出：当前指令地址 PC_Out。

模块原理：在时钟上升沿，若 Reset=0，初始化 PC 为 0。在 Reset 不为 0 时，若 PCWre=0，PC 不更新，否则 PC 更新。

9. PCNew (PC 更新模块)

```

module CPU2_PCNew(
    input [31:0]PC_Out,
    input [15:0]immediate,
    input [25:0]addr,
    input [31:0]jump,
    input wire [1:0]PCSrc,
    input wire ExtSel,
    output reg[31:0]PC_In
);
initial begin
    PC_In=0;
end
reg [31:0]Extend_Result;
reg [31:0]Extend_Address;
reg [31:0]PC4;
always@(*)
begin
    if(ExtSel==0) Extend_Result<={{16{1'b0}},immediate};
    else begin
        if(immediate[15]==0) Extend_Result<={{16{1'b0}},immediate};
        else begin
            Extend_Result[31:16]<=16'b1111_1111_1111_1111;
            Extend_Result[15:0]<=immediate;
        end
    end
    Extend_Address<=Extend_Result*4;
    PC4=PC_Out+4;
    case(PCSrc)
    2'b00:PC_In<=PC_Out+4;
    2'b01:PC_In<=PC_Out+4+Extend_Address;
    2'b10:PC_In<=32'b0000_0000_0000_0000_0000_0010_0000;
    2'b11:PC_In<={PC4[31:28],addr,1'b0,1'b0};
    endcase
end
endmodule

```

模块输入：当前 PC，立即数，跳转地址（指 addr，jump 已经不使用了），PC 更新控制信号 PCSrc，扩展控制信号 ExtSel。

模块输出：下一个 PC。

模块原理：对于每一个输入的 PC，若 PCSrc=0，PC+4；若 PCSrc=1，PC+4+立即数扩展后的结果*4；若 PCSrc=2，跳转。若 PCSrc=3，跳转。

10. 写寄存器控制模块（RegData）

```

module CPU2_RegData(
    input [31:0]PC_Out,
    input [31:0]Data_Out,
    input WrRegDSrc,
    output reg[31:0]WriteData
);
    always@(*)begin
        if(WrRegDSrc==0) WriteData<=PC_Out+4;
        else WriteData<=Data_Out;
    end
endmodule

```

模块输入：当前的 PC，数据存储器的输出和 ALU 运算结果选择后的输出 DataOut，控制信号 WrRegDSrc

模块输出：要写入的值。

模块原理：若 WrRegDSrc 为 0，写入 PC+4。否则，写入 DataOut。

11. 寄存器组 (Register)


```

module CPU2_Register(
    input CLK,
    input wire Reset,
    input wire RegWre,
    input wire [4:0]rs,
    input wire [4:0]rt,
    input wire [4:0]rd,
    input wire [1:0]RegDst,
    input wire [31:0]WriteData,
    output [31:0]Read_Data1,
    output [31:0]Read_Data2
);

    integer i;
    reg [31:0] file[0:31];
    initial begin
        for(i=0;i<32;i=i+1)begin
            file[i] = 32'b0000_0000_0000_0000_0000_0000_0000_0000;
        end
    end

    assign Read_Data1 =(rs)?file[rs]:0;
    assign Read_Data2 =(rt)?file[rt]:0;

    always@(negedge CLK)begin
        if(RegWre!=0)
            if(RegDst==0)file[31] <= WriteData;
            else if(RegDst==1)file[rt]<=WriteData;
            else file[rd]<=WriteData;
        end
    end
endmodule

```

模块输入：CLK, Reset, 寄存器组写控制信号 RegWre, 写寄存器组地址来源控制信号 RegDst, 写入数据 WriteData, 指令的 rs, rt, rd 字段。

模块输出：寄存器组读出数据。

模块原理：寄存器组的两个读出数据分别读出 rs, rt 地址处的数据。

若 RegWre != 0, 在时钟下降沿写入。写入时, 若 RegDst=0, 写到地址 31;

若 RegDst=1, 写到 rt; 若 RegDst=2, 写到 rd。

12. 数据输出选择 (DataSelect)

```

module CPU2_DataSelect(
    input wire [31:0]result_out,
    input wire [31:0]Datamem,
    input wire DBDataSrc,
    output wire [31:0]Data_Out
);
    assign Data_Out =(DBDataSrc)? Datamem : result_out;
endmodule

```

模块输入：ALU 寄存器输出，数据存储器输出，控制信号。

模块输出：DataOut。

模块原理：若 DBDataSrc=1，输出数据存储器输出。否则，输出 ALU 寄存器输出。

13. CPU 顶层模块：

```

22
23 module CPU2_Top(
24     input wire CLK,
25     input wire Reset,
26     output wire PCWre,
27     output wire ALUSrcA,
28     output wire ALUSrcB,
29     output wire mRD,
30     output wire mWR,
31     output wire DBDataSrc,
32     output wire ExtSel,
33     output wire [1:0]PCSrc,
34     output wire InsMemRW,
35     output wire [1:0]RegDst,
36     output wire RegWre,
37     output wire [2:0]ALUOp,
38     output wire [31:0]Ins_Out,
39     output wire [31:0]PC_Out,
40     output wire zero,
41     output wire [31:0]result_out,
42     output wire [31:0]data1,
43     output wire [31:0]data2,
44     output wire [31:0]Read_Data1,
45     output wire [31:0]Read_Data2,
46     output wire [31:0]PC_In,
47     output wire [31:0]Data_OutR,
48     output wire [4:0]rs,
49     output wire [4:0]rt,
50     output wire [4:0]rd,
51     output wire [4:0]sa,
52     output wire [5:0]op,
53     output wire [10:0]func,
54     output wire sign,
55     output wire [15:0]immediate,
56     output wire [25:0]addr,
57     output wire IRWre,
58     output wire WrRegDSrc,
59
60     output wire [31:0]WriteReg,
61     output wire [2:0]cur_state,
62     output wire [31:0]result,
63     output wire [5:0]sig
64 );

```

```

○ assign op = Ins_Out[31:26];
○ assign rs = Ins_Out[25:21];
○ assign rt = Ins_Out[20:16];
○ assign rd = Ins_Out[15:11];
○ assign func = Ins_Out[10:0];
○ assign sa = Ins_Out[10:6];
○ assign immediate = Ins_Out[15:0];
○ assign addr = Ins_Out[25:0];
○ assign sig = Ins_Out[5:0];

wire [31:0] jump;
wire [31:0] IDataOut;
wire [31:0] WriteData;
wire [31:0] Read_ADR;
wire [31:0] Read_BDR;
wire [31:0] Datamem;
wire [31:0] Data_Out;

```

模块输入: CLK,Reset。

模块输出: 为调试方便, 我将所有的控制信号都放在了模块输出中。实际上, 在 Basys 顶层模块中需要的输出, 只有当前 PC, 下一个 PC, ALU 寄存器输出, 数据存储寄存器输出, 两个寄存器读结果, rs, rt。

模块原理: 在顶层模块中, 先将指令分解成需要的字段, 然后实例化各个模块, 将所有模块连接起来, 形成一个 CPU。

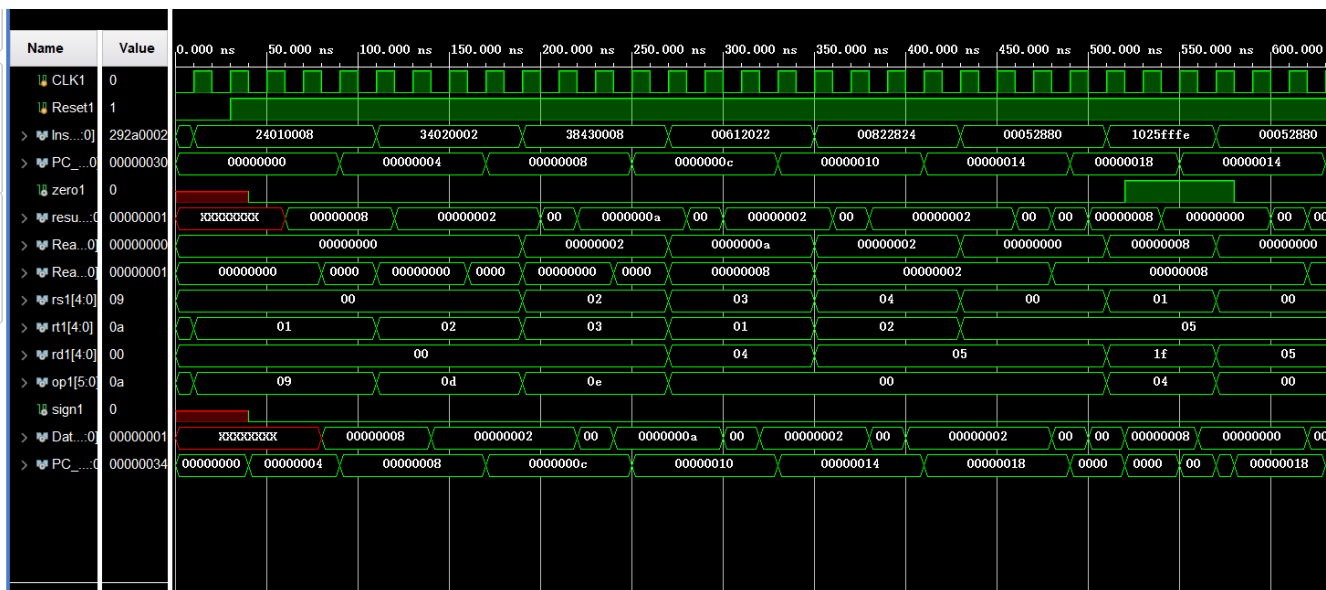
CPU 验证 (波形结果与说明):

Result: ALU 寄存器输出。

DataOut: ALU 寄存器与数据寄存器输出选择后的结果

ReadData1: file[rs]的值

ReadData2: file[rt]的值



第一条指令：0-90ns

addiu \$1,\$0,8

10-30ns: IF 周期，取指令，指令寄存器输出更新为 24010008

30-50ns: ID 周期，指令译码，由于此时寄存器各个地址的值都为 0，故两个 readdata 没有发生变化

50-70ns: EX 周期，执行指令。60ns 的时钟下降沿 ALU 寄存器输出结果更新为 8。

70-90ns: WB 周期，写回寄存器，file[1]更新为 8。

第二条指令：90-170ns

ori \$2,\$0,2

90-110ns: 取指令，指令寄存器输出更新为 34020002。

110-130ns: 指令译码。

130-150ns: EX 周期。此时，Result 为 2。

150-170ns: WB 周期。160ns 时，file[2]更新为 2。

第三条指令：170-250ns

xori \$3,\$2,8

170-190ns: IF 周期，指令存储器输出更新为 38430008。

190-210ns: ID 周期。ReadData1 更新为 2，ReadData2 更新为 0。

210-230ns: EX 周期。Result 更新为 10。

230-250ns: WB 周期。File[3]更新为 10。

第四条指令：250-330ns

sub \$4,\$3,\$1

250-270ns: IF 周期，指令存储器输出更新为 00612022。

270-290ns: ID 周期。ReadData1 更新为 10，ReadData2 更新为 8。

290-310ns: EX 周期。Result 更新为 2。

310-330ns: WB 周期。由于两个 ReadData 分别指出了 rs 和 rt 处的值，故 File[4]无法读出。

第五条指令：330-410ns。

and \$5,\$4,\$2

330-350ns: IF 周期，指令存储器输出更新为 00822824。

350-370ns: ID 周期。ReadData1 更新为 2，ReadData2 更新为 2。

370-390ns: EX 周期。Result 更新为 2。

390-410ns: WB 周期。由于两个 ReadData 分别指出了 rs 和 rt 处的值，故 File[5]无法读出。

第六条指令：410-490ns。

sll \$5,\$5,2

410-430ns: IF 周期，指令存储器输出更新为 00052880。

430-450ns: ID 周期。ReadData1 更新为 0 (sll 指令 rs 字段为 0)，ReadData2 不变。

450-470ns: EX 周期。Result 更新为 8。

470-490ns: WB 周期。File[5]更新为 8。

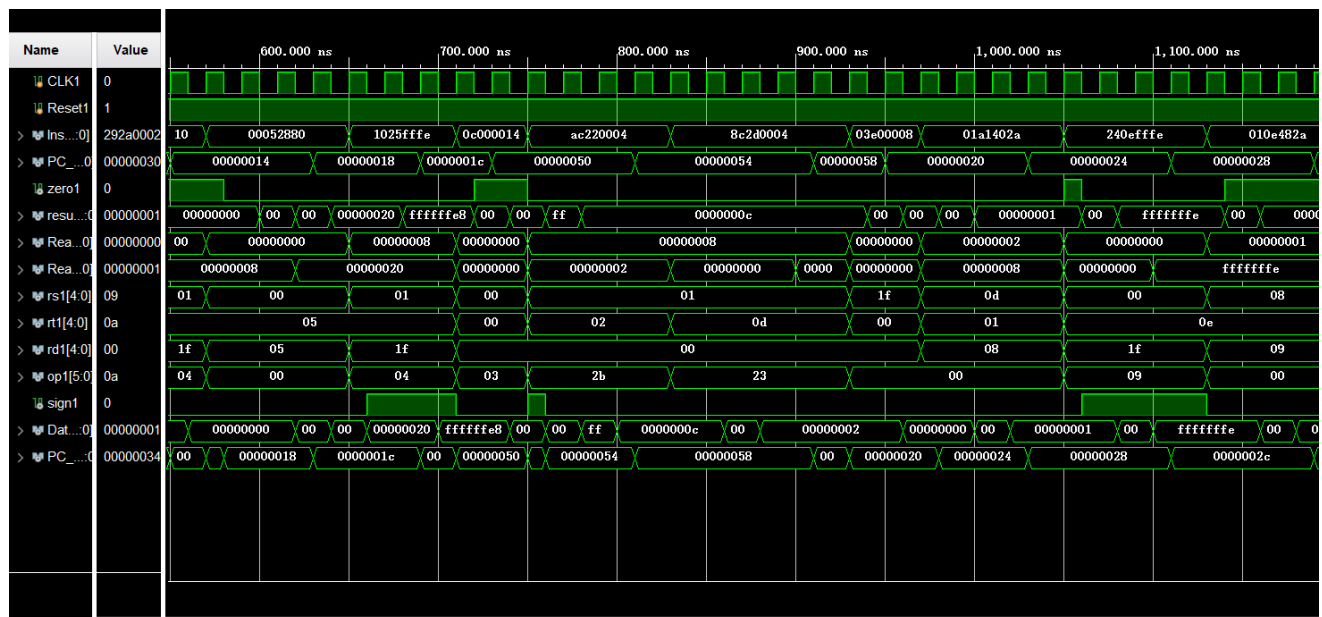
第七条指令: 490-550ns。

beq \$5,\$1,-2

490-510ns: IF 周期, 指令存储器输出更新为 10A1FFFE。

510-530ns: ID 周期。ReadData1 更新为 8 (sll 指令 rs 字段为 0), ReadData2 为 8 (上一条指令执行过程中已经更新)。

530-550ns: EX 周期。Zero=1, 跳转到 PC=14 处, 执行 sll 指令。



第八条指令: 550-630ns。

sll \$5,\$5,2

550-570ns: IF 周期, 指令存储器输出更新为 00052880。

570-590ns: ID 周期。ReadData1 更新为 0 (sll 指令 rs 字段为 0), ReadData2 不变 (=8)。

590-610ns: EX 周期。Result 更新为 32。

610-630ns: WB 周期。File[5]更新为 32。

第九条指令: 630-690ns

beq \$5,\$1,-2

630-650ns: IF 周期, 指令存储器输出更新为 10A1FFFE。

650-670ns: ID 周期。ReadData1 更新为 32 (sll 指令 rs 字段为 0), ReadData2 更新为 8。

670-690ns: EX 周期。Zero=0, PC+4, 执行 jal 指令。

第十条指令: 690-730ns

jal 0x00000050

690-710ns: IF 周期。指令寄存器输出更新为 0C000014。

710-730ns: ID 周期, 下一条指令地址更新为 00000050。

第十一条指令：730-810ns

sw \$2,4(\$1)

730-750ns: IF 周期，指令存储器输出更新为 AC220004。

750-770ns: ID 周期。ReadData1 更新为 8，ReadData2 更新为 2。

770-790ns: EX 周期。Result 更新为 12 (8+4)。

790-810ns: MEM 周期。将 file[2]处的值存入数据存储器的地址 12 处。

第十二条指令：810-910ns

lw \$13,4(\$1)

810-830ns: IF 周期，指令存储器输出更新为 BC2D0004。

830-850ns: ID 周期。ReadData1 依然为 8，ReadData2 更新为 0。

850-870ns: EX 周期。Result 依然为 12 (8+4)。

870-890ns: MEM 周期。将数据存储器地址 12 处的值取出。

890-910ns: WB 周期。将取出的值放到 file[13]处。在 900ns 的下降沿，file[13]更新为 2

第十三条指令：910-950ns

jr \$31

910-930ns: IF 周期。指令寄存器输出更新为 00F80008。

930-950ns: ID 周期，下一条指令地址更新为 00000020。

第十四条指令：950-1030ns

slt \$8,\$13,\$1

550-570ns: IF 周期，指令存储器输出更新为 01A1202A。

570-590ns: ID 周期。ReadData1 为 2，ReadData2 更新为 8。

590-610ns: EX 周期。2<8, Result 更新为 1。

610-630ns: WB 周期。File[8]更新为 1。

第十五条指令：1030-1110ns

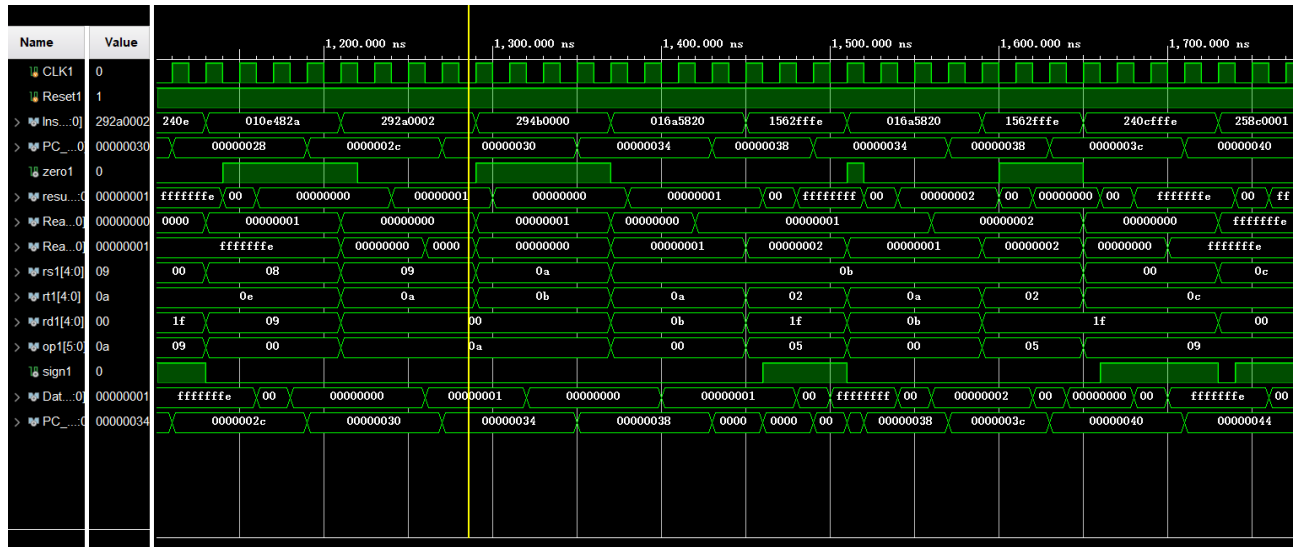
addiu \$14,\$0,-2

1030-1050ns: IF 周期，指令存储器输出更新为 240EFFFFE。

1050-1070ns: ID 周期。ReadData1(file[0])为 0，ReadData2(file[14])为 0。

1070-1090ns: EX 周期。Result 更新为 0+(-2)=-2。

1090-1110ns: WB 周期。File[14]更新为-2。



第十六条指令：1110-1190ns

slt \$9,\$8,\$14

1110-1130ns: IF 周期，指令存储器输出更新为 008E482A。

1130-1150ns: ID 周期。ReadData1(file[8])更新为 0，ReadData2(file[14])更新为-2。

1150-1170ns: EX 周期。0>-2, Result 更新为 0。

1170-1190ns: WB 周期。File[9]更新为 0。File[rd]的值在仿真上看不出来。

第十七条指令：1190-1270ns

slti \$10,\$9,2

1190-1210ns: IF 周期，指令存储器输出更新为 292A0002。

1210-1230ns: ID 周期。ReadData1(file[9])为 0，ReadData2(file[10])为 0。

1230-1250ns: EX 周期。0<2, Result 更新为 1。

1250-1270ns: WB 周期。File[10]更新为 1。

第十八条指令：1270-1350ns

slti \$11,\$10,0

1190-1210ns: IF 周期，指令存储器输出更新为 294B0000。

1210-1230ns: ID 周期。ReadData1(file[10])为 1，ReadData2(file[11])为 0。

1230-1250ns: EX 周期。1>0, Result 为 0。

1250-1270ns: WB 周期。File[11]不变。

第十九条指令：1350-1430ns

add \$11,\$11,\$10

1190-1210ns: IF 周期，指令存储器输出更新为 016A5820。

1210-1230ns: ID 周期。ReadData1(file[11])为 0，ReadData2(file[10])为 1。

1230-1250ns: EX 周期。Result = 0+1=1

1250-1270ns: WB 周期。File[11]更新为 1（rs 和 rd 字段相等，因此可以从仿真上看起来变化）。

第二十条指令：1430-1490ns

bne \$11,\$2,-2

1430-1450ns: IF 周期，指令存储器输出更新为 1562FFFE。

1450-1470ns: ID 周期。ReadData1(file[11])为 1，ReadData2(file[2])更新为 2。

1470-1490ns: EX 周期。zero=0，跳转到 00000034。

第二十一条指令：1490-1570ns

add \$11,\$11,\$10

1490-1510s: IF 周期，指令存储器输出更新为 016A5820。

1510-1530ns: ID 周期。ReadData1(file[11])为 1，ReadData2(file[10])为 1。

1530-1550ns: EX 周期。Result = 1+1=2

1550-1570ns: WB 周期。File[11]更新为 2（rs 和 rd 字段相等，因此可以从仿真上看出变化）。

第二十二条指令：1570-1630ns

bne \$11,\$2,-2

1570-1590ns: IF 周期，指令存储器输出更新为 1562FFFE。

1590-1610ns: ID 周期。ReadData1(file[11])为 2，ReadData2(file[2])为 2。

1610-1630ns: EX 周期。zero=1，跳转到 0000003C。

第二十三条指令：1630-1710

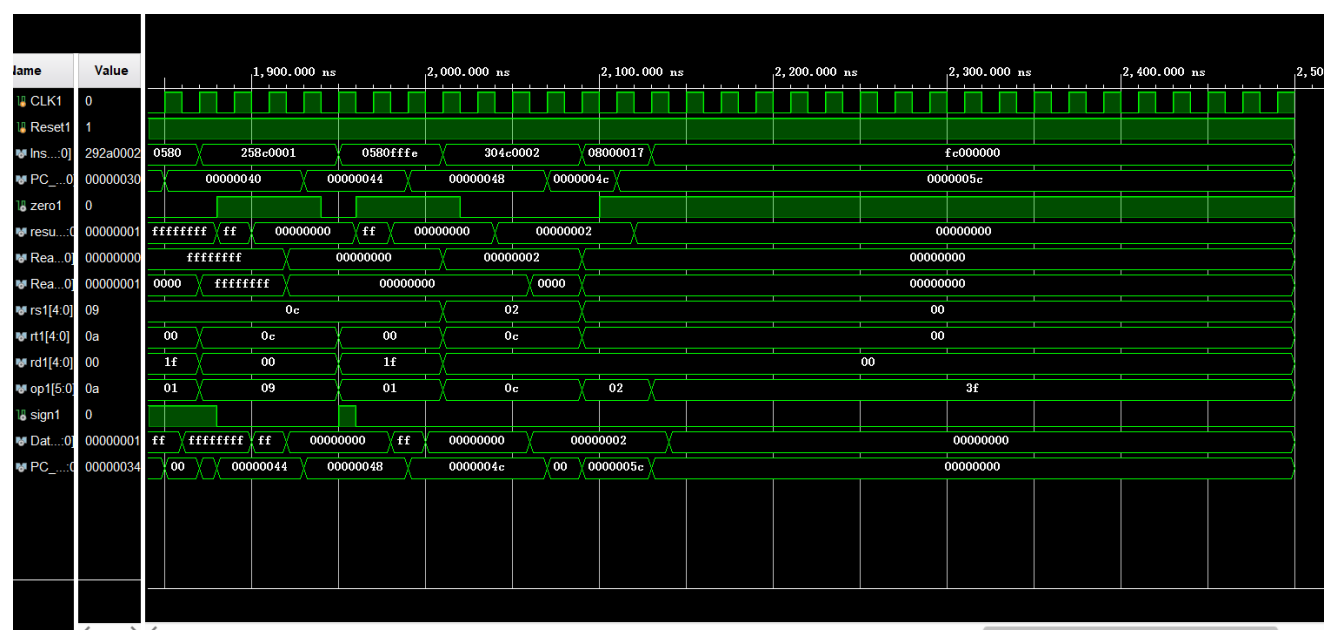
addiu \$12,\$0,-2

1630-1650s: IF 周期，指令存储器输出更新为 240CFFFE。

1650-1670ns: ID 周期。ReadData1(file[0])为 0，ReadData2(file[12])为 0。

1670-1690ns: EX 周期。Result = 0-2=-2

1690-1710ns: WB 周期。File[12]更新为-2。



第二十四条指令：1710-1790ns

addiu \$12,\$12,1

1710-1730ns: IF 周期, 指令存储器输出更新为 258C0001。

1730-1750ns: ID 周期。ReadData1(file[12])为-2, ReadData2(file[12])为-2。

1750-1770ns: EX 周期。Result = -2+1=-1

1770-1790ns: WB 周期。File[12]更新为-1。

第二十五条指令: 1790-1850ns

bltz \$12,-2

1790-1810ns: IF 周期, 指令存储器输出更新为 0580FFFE。

1810-1830ns: ID 周期。ReadData1(file[12])为-1, ReadData2(file[0])为 0。

1830-1850ns: EX 周期。-1<0, 跳转到 00000040

第二十六条指令: 1850-1930ns

addiu \$12,\$12,1

1850-1870ns: IF 周期, 指令存储器输出更新为 258C0001。

1870-1890ns: ID 周期。ReadData1(file[12])为-1, ReadData2(file[12])为-1。

1890-1910ns: EX 周期。Result = -1+1=0

1910-1930ns: WB 周期。File[12]更新为 0。

第二十七条指令: 1930-1990ns

bltz \$12,-2

1930-1950ns: IF 周期, 指令存储器输出更新为 0580FFFE。

1950-1970ns: ID 周期。ReadData1(file[12])为 0, ReadData2(file[0])为 0。

1970-1990ns: EX 周期。0>=0, 跳转到 00000048

第二十八条指令: 1990-2070ns

andi \$12,\$2,2

1990-2010ns: IF 周期, 指令存储器输出更新为 304C0002。

2010-2030ns: ID 周期。ReadData1(file[2])为 2, ReadData2(file[12])为 0。

2030-2050ns: EX 周期。Result = 2 & 2 = 2

2050-2070ns: WB 周期。File[12]更新为 2。

第二十九条指令: 2070-2110ns

j 0x000005C

2070-2090ns: IF 周期, 指令存储器输出更新为 08000017。

2090-2110ns: ID 周期, 下一条指令地址更新为 0000005C。

第三十条指令：2110ns-仿真结束

halt

2110ns-2130ns: IF 周期，指令存储器输出更新为 FC000000

2130-2150ns: ID 周期，所有输出清零。

2150ns-仿真结束：所有输出不变。我在设计时将 halt 的下一条指令设置为了 0。

CPU 实现（Basys 模块设计）：

1. 消抖模块：

```
23 module CPU2_Bounce(  
24     input clk,  
25     input button,  
26     output signal  
27 );  
28     reg [1:0]clkreg;  
29     reg clkslow;  
30     parameter times=5000;  
31     reg [31:0]count;  
32  
33     initial begin  
34         clkreg=0;  
35         count=0;  
36     end  
37  
38     always@(posedge clk)  
39     begin  
40         if(count<times)  
41             count<=count+1;  
42         else  
43             begin  
44                 count<=0;  
45                 clkslow=~clkslow;  
46             end  
47     end  
48  
49     always@(posedge clkslow)  
50     begin  
51         clkreg[0]<=button;  
52         clkreg[1]<=clkreg[0];  
53     end  
54  
55     assign signal = !((clkreg[1]&clkreg[0])|(~clkreg[1]&clkreg[0]));  
56 endmodule
```

由于 Basys 板上的按键为机械按键，按下时可能发生抖动，故需要消抖模块，使得按键稳定后再产生脉冲。

2. 显示选择模块

```

module CPU2_Display(
    input signal,
    input [1:0]SW,
    input [3:0]ctr,
    input [31:0]PC_Out,
    input [31:0]PC_In,
    input [4:0]rs,
    input [4:0]rt,
    input [31:0]Read_Data1,
    input [31:0]Read_Data2,
    input [31:0]Data_Out,
    input [31:0]result,
    output reg[3:0]digit
);
    always@(signal)
    begin
        case(ctr)
            4'b1110:
                begin
                    case(SW)
                        2'b00:digit <= PC_In[3:0];
                        2'b01:digit <= rs[3:0];
                        2'b10:digit <= rt[3:0];
                        2'b11:digit <= Data_Out[3:0];
                    endcase
                end
            4'b1101:
                begin
                    case(SW)
                        2'b00:digit <= PC_In[7:4];
                        2'b01:digit <= {3'b000,rs[4]};
                        2'b10:digit <= {3'b000,rt[4]};
                        2'b11:digit <= Data_Out[7:4];
                    endcase
                end
            4'b1011:

```

```

        end
4'b1011:
    begin
        case(SW)
            2'b00:digit <= PC_Out[3:0];
            2'b01:digit <= Read_Data1[3:0];
            2'b10:digit <= Read_Data2[3:0];
            2'b11:digit <= result[3:0];
        endcase
    end
4'b0111:
    begin
        case(SW)
            2'b00:digit <=PC_Out[7:4];
            2'b01:digit <= Read_Data1[7:4];
            2'b10:digit <= Read_Data2[7:4];
            2'b11:digit <= result[7:4];
        endcase
    end
endcase
end
endmodule

```

在选择模块中，根据 SW 输入的不同，在数码管上显示不同的内容。若 SW=0，显示当前 PC 和下一 PC，若 SW=1，显示 rs 寄存器和其中的值，若 SW=2，显示 rt 寄存器和其中的值，若 SW=3，显示 ALU 输出和数据存储器输出。

3. 数码管显示模块

```

20 //////////////////////////////////////////////////
21
22
23 module CPU2_Led(
24     input reset,
25     input [3:0]digit,
26     output reg[7:0]light
27 );
28     always@(digit or reset)
29     begin
30         if(reset==0) light=8'b1111110; //.
31         else begin
32             case(digit)
33                 4'b0000:light=8'b00000011; //0
34                 4'b0001:light=8'b10011111; //1
35                 4'b0010:light=8'b00100101; //2
36                 4'b0011:light=8'b00001101; //3
37                 4'b0100:light=8'b10011001; //4
38                 4'b0101:light=8'b01001001; //5
39                 4'b0110:light=8'b01000001; //6
40                 4'b0111:light=8'b00011111; //7
41                 4'b1000:light=8'b00000001; //8
42                 4'b1001:light=8'b00011001; //9
43                 4'b1010:light=8'b00010001; //A
44                 4'b1011:light=8'b11000001; //b
45                 4'b1100:light=8'b01100011; //C
46                 4'b1101:light=8'b10000101; //d
47                 4'b1110:light=8'b01100001; //E
48                 4'b1111:light=8'b01110001; //F
49                 default:light=8'b11111100; //-
50             endcase
51         end
52     end
53
54 endmodule
55

```

模块输入：要显示的内容，reset。

输出：数码管信号。

模块原理：若 Reset=0,数码管显示 “....”，否则数码管显示对应的内容。

4. Basys 顶层模块：

```

2
3 module CPU2_Basys(
4     input button,
5     input [1:0]SW,
6     input CLK,    //Basys3's Clock
7     input Reset,
8     output [7:0]light,
9     output reg[3:0]ctr
10 );
11
12     wire [31:0]PC_Out;
13     wire [31:0]PC_In;
14     wire [31:0]result;
15     wire [31:0]Read_Data1;
16     wire [4:0]rs;
17     wire [4:0]rt;
18     wire [31:0]IDataOut;
19     wire [31:0]Data_Out;
20     wire [31:0]Read_Data2;
21     wire signal;
22
23     parameter T1MS = 5000;
24     reg [31:0]count;
25     wire [3:0]digit;
26

```

```

CPU2_Top cpu(
    .CLK(signal),
    .Reset(Reset),
    .Ins_Out(IDataOut),
    .rs(rs),
    .rt(rt),
    .PC_Out(PC_Out),
    .Data_OutR(Data_Out),
    .result(result),
    .PC_In(PC_In),
    .Read_Data1(Read_Data1),
    .Read_Data2(Read_Data2)
);

```

```

initial begin
    count <= 0;
    ctr <= 4'b0111;
end

```

```

CPU2_Bounce bounce(
    .clk(CLK),
    .button(button),
    .signal(signal)
);

```

```

always@(posedge CLK)
begin
    if(Reset==0) begin
        count<=0;
        ctr<=4'b0000;
    end
    else begin
        count=count+1;
        if(count==T1MS)begin
            count<=0;
            case(ctr)
                4'b1110:ctr <= 4'b1101;
                4'b1101:ctr <= 4'b1011;
                4'b1011:ctr <= 4'b0111;
                4'b0111:ctr <= 4'b1110;
                4'b0000:ctr <= 4'b0111;
            endcase
        end
    end
end

CPU2_Led My_led(
    .reset(Reset),
    .digit(digit),
    .light(light)
);

CPU2_Display My_Display(
    .signal(signal),
    .ctr(ctr),
    .SW(SW),
    .digit(digit),
    .PC_In(PC_In),
    .result(result),
    .PC_Out(PC_Out),
    .rs(rs),
    .rt(rt),
    .Data_Out(Data_Out),
    .Read_Data1(Read_Data1),
    .Read_Data2(Read_Data2)
);
endmodule

```

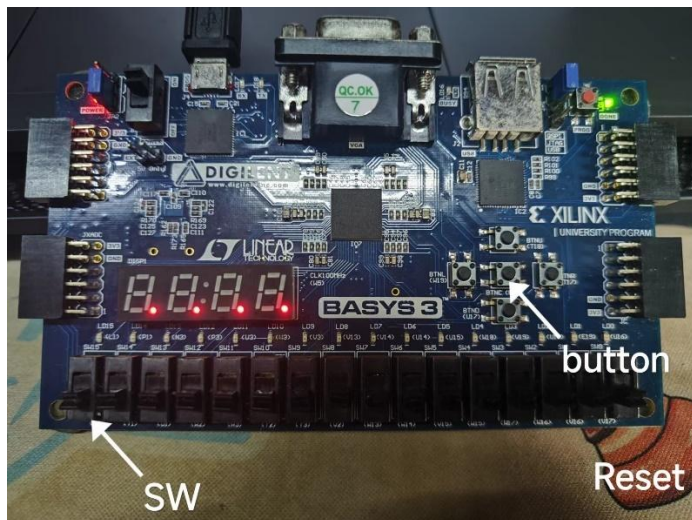
模块输入：手动脉冲 button，reset，CLK，数码管显示选择 SW

模块输出：数码管显示结果 light，数码管位选 ctr。

模块原理：每产生一个手动脉冲，模块生成对应的 ctr 信号。顶层模块同时将所有模块连接起来，成为一个整体。

CPU 实现（在 Basys3 实验板上运行）：

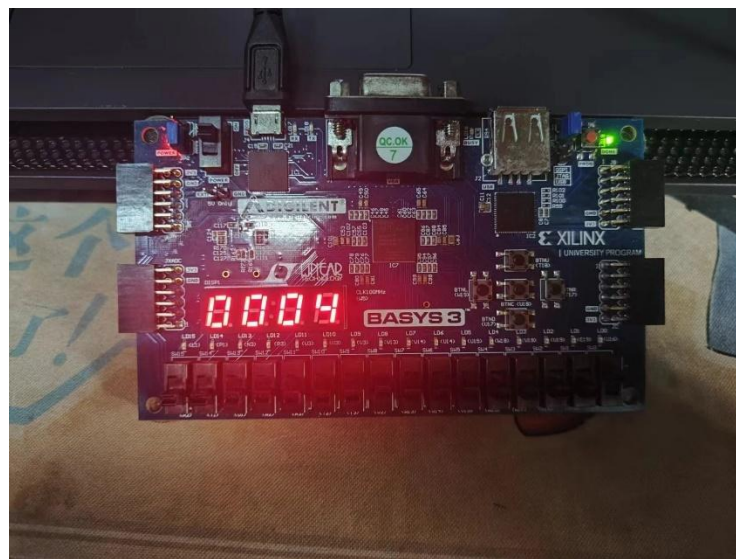
端口映射：



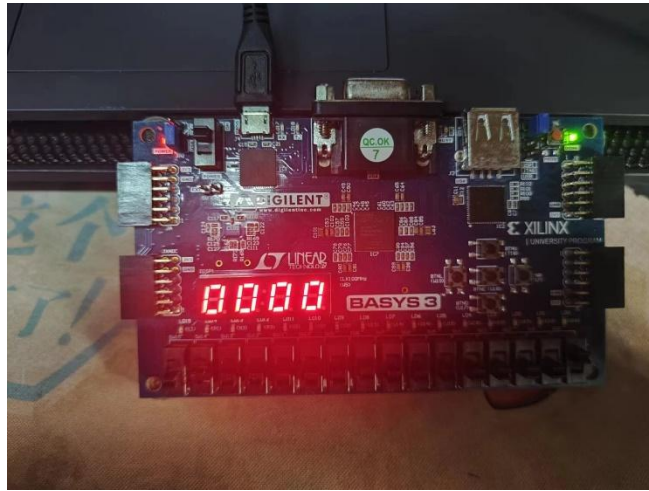
操作方法： 按下 button，产生一个脉冲，对应一个时钟周期。拨动 SW，调整数码管显示内容。拨动 Reset，重置 CPU。如果一条指令需要四个时钟周期来执行，那么按动 button 四次后，跳转到下一条指令。其余需要三个、两个、五个时钟周期的指令同理。

第一条指令：addiu \$1,\$0,8

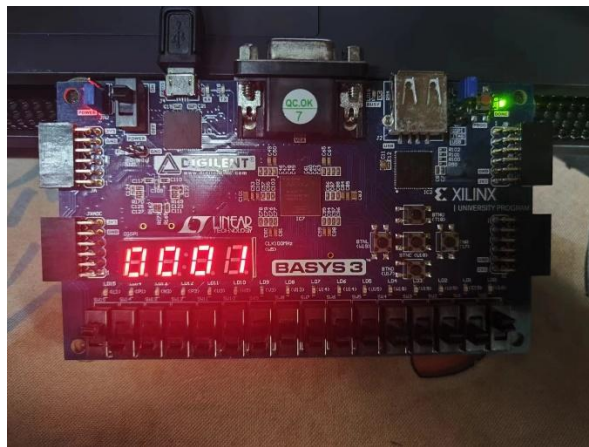
当前指令地址为 00，下一条指令地址为 04.



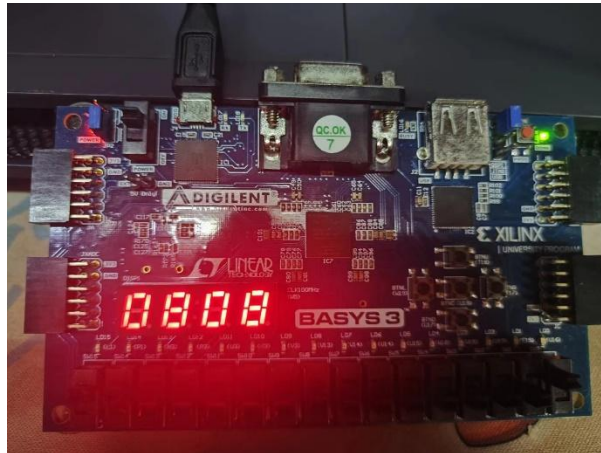
0 号寄存器的值为 0.



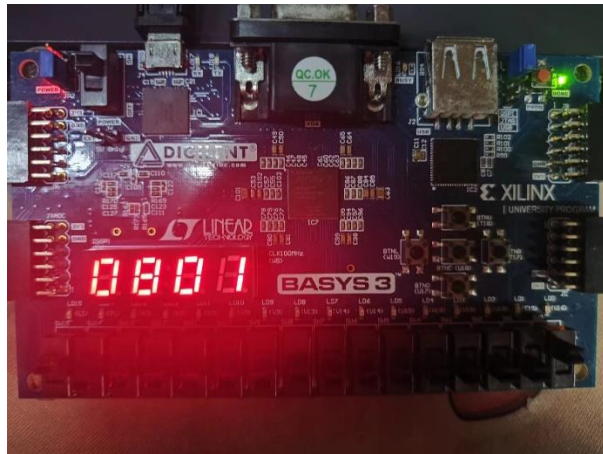
1 号寄存器的值为 1.



ALU 和数据输出均为 8.

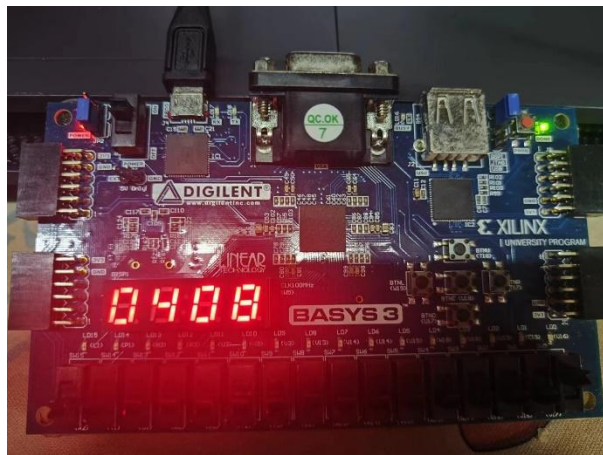


结果写入 1 号寄存器，1 号寄存器值为 8.

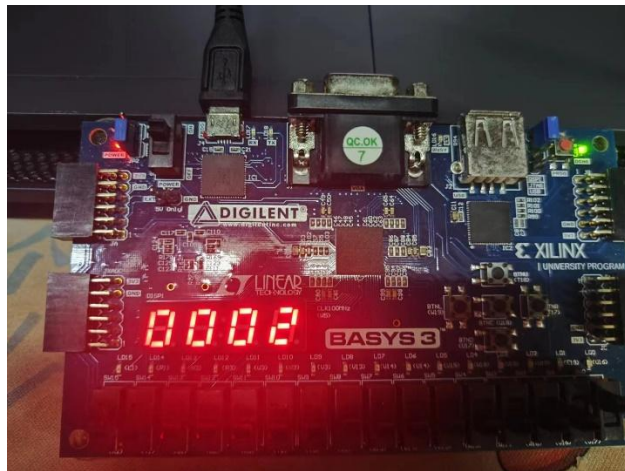


第二条指令: `ori $2,$0,2`

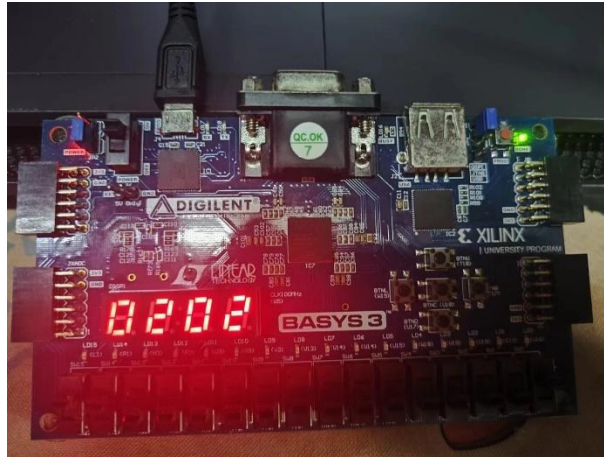
当前指令地址为 04, 下一条指令地址为 08.



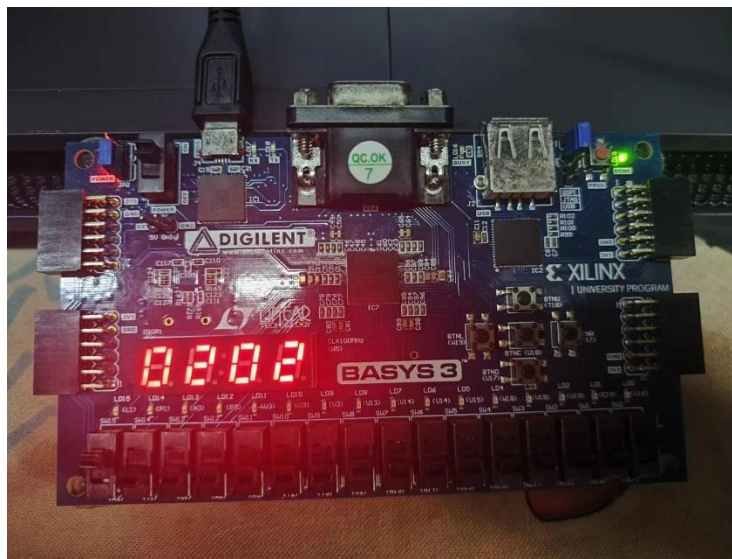
2 号寄存器的值为 0.



ALU 和数据输出均为 2.

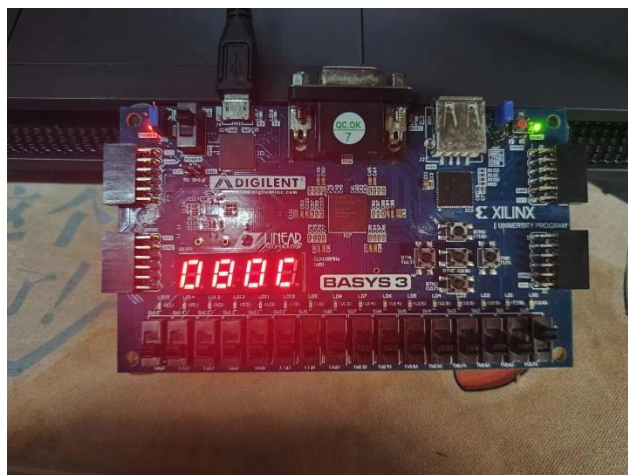


结果写入 2 号寄存器，2 号寄存器的值为 2.

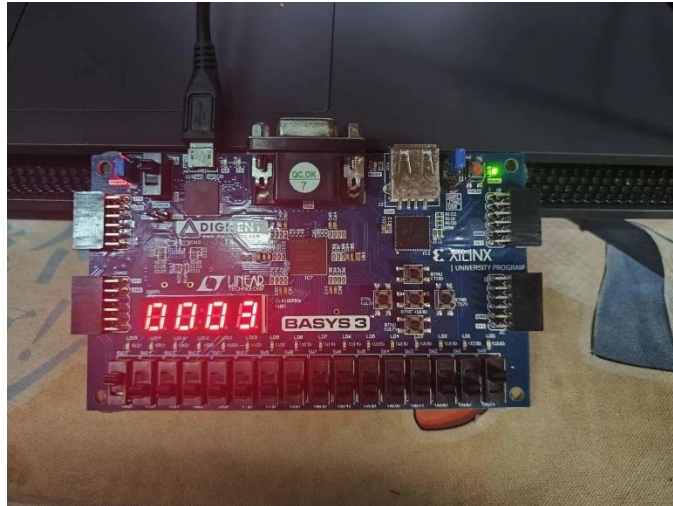


第三条指令: `xori $3,$2,8`

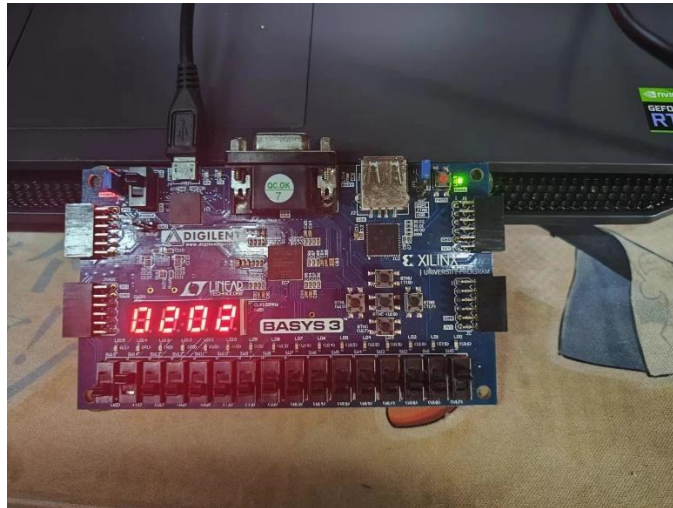
当前指令地址为 08，下一条指令地址为 0C。



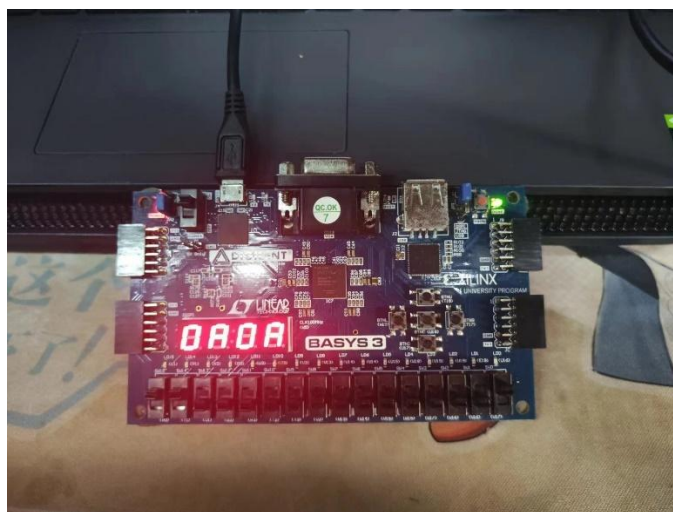
3 号寄存器的值为 0.



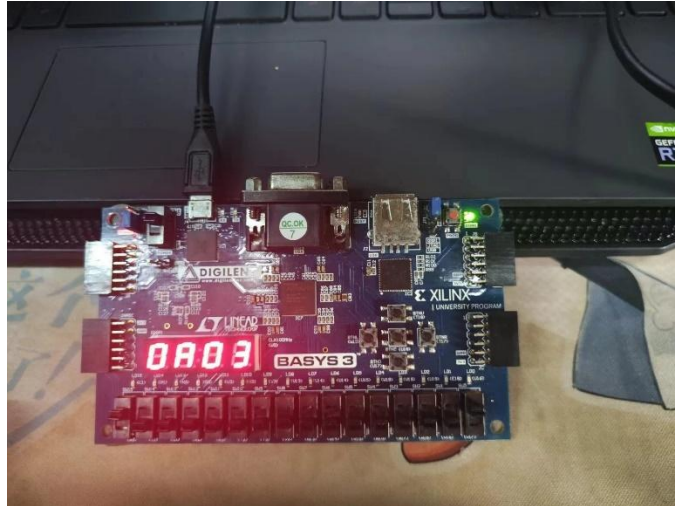
2 号寄存器的值为 2.



ALU 和数据输出都是 10.

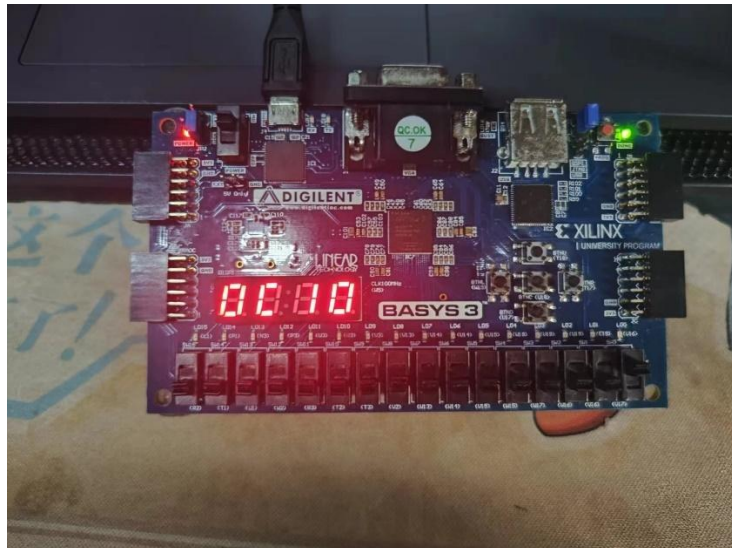


结果写入 3 号寄存器，3 号寄存器值为 10.

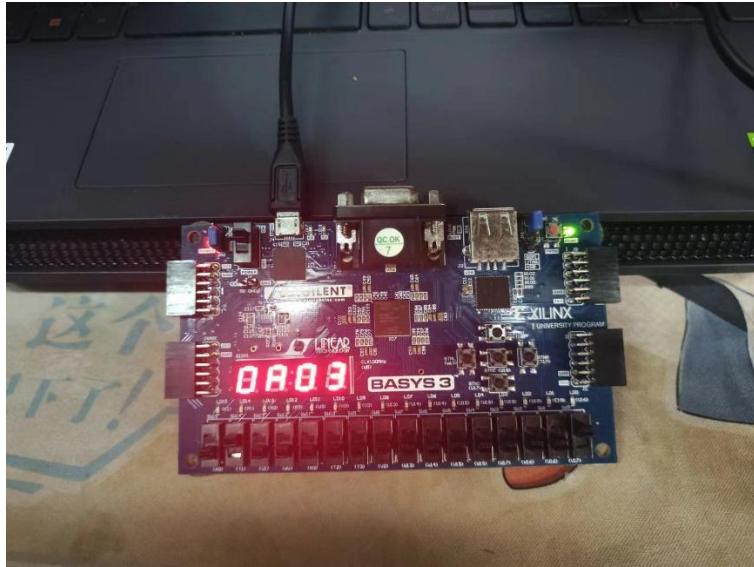


第四条指令: `sub $4,$3,$1`

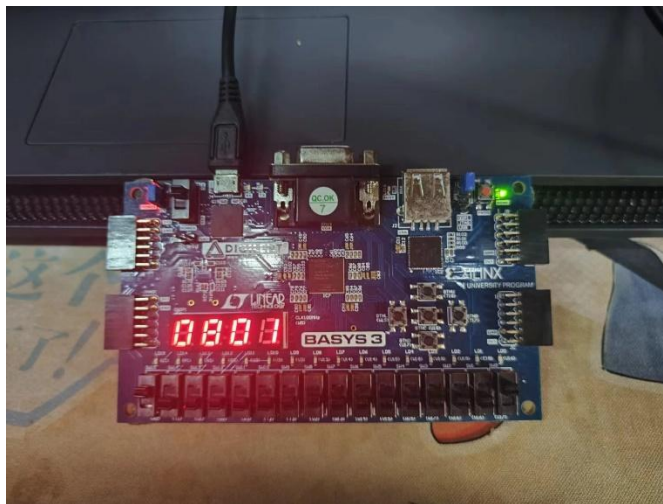
当前指令地址: 0C 下一条指令地址: 10



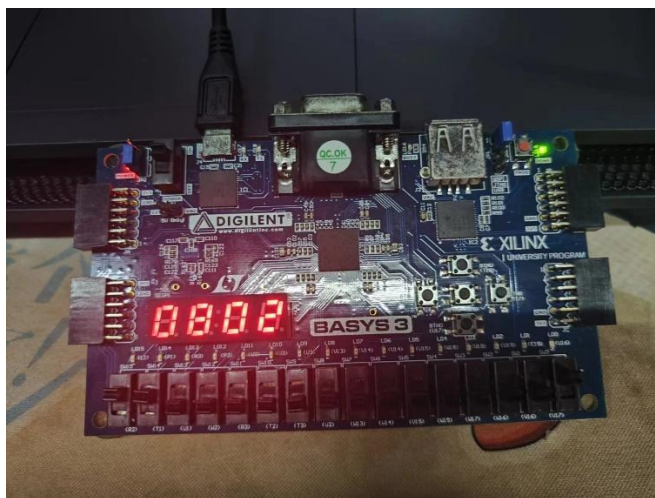
3 号寄存器的值为 10



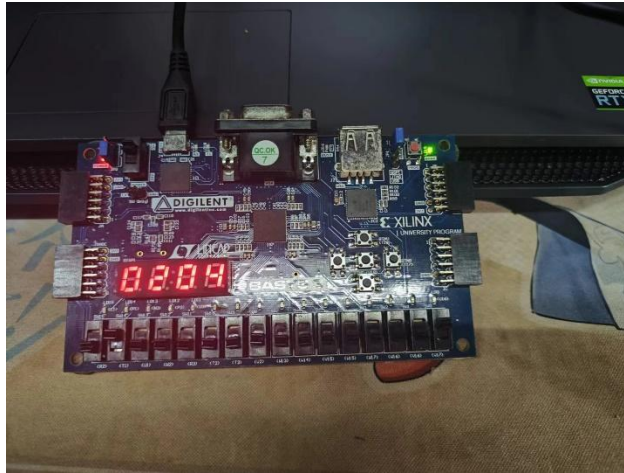
1 号寄存器的值为 8.



ALU 结果为 2.

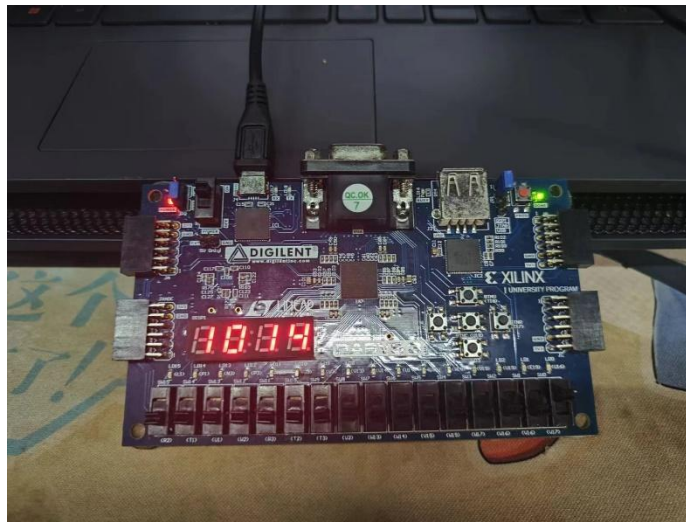


结果写入 4 号寄存器，4 号寄存器的值为 2.

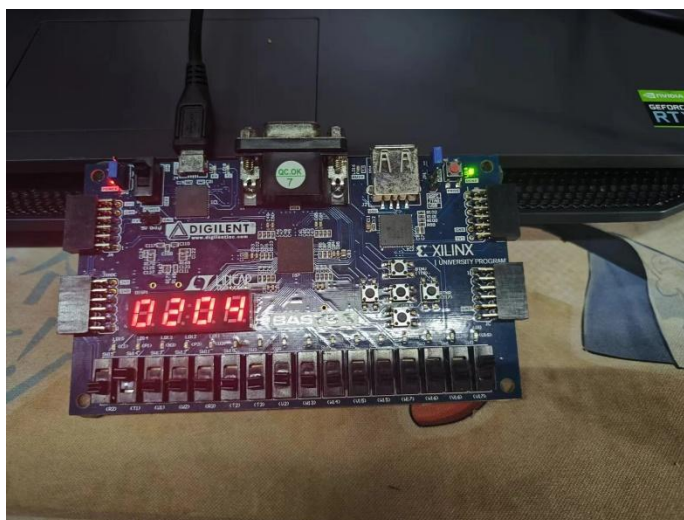


第五条指令: and \$5,\$4,\$2

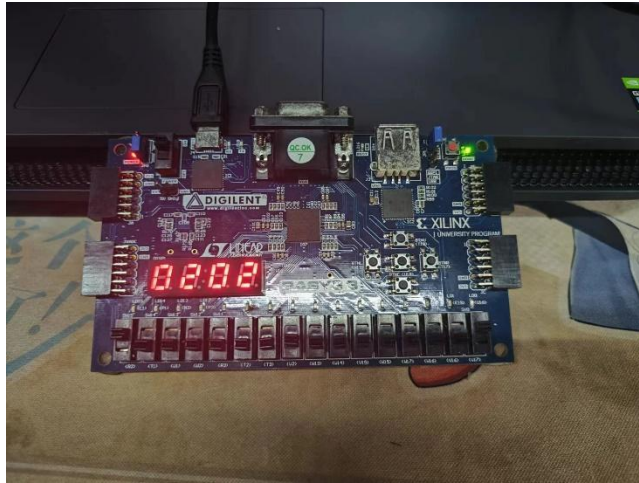
当前指令地址: 10 下一条指令地址: 14



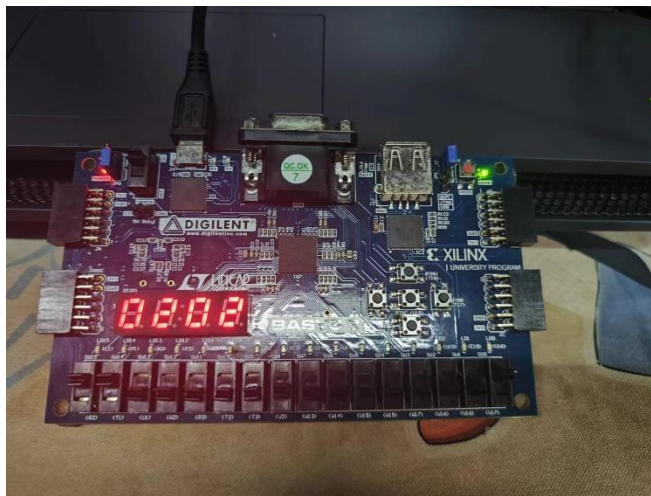
4 号寄存器的值为 2



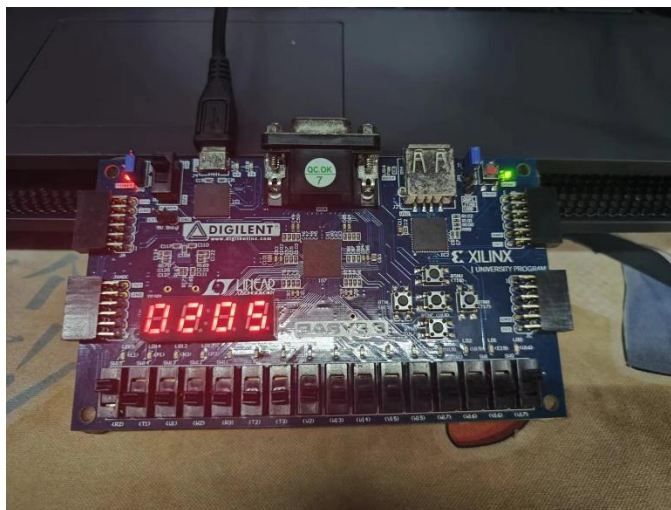
2 号寄存器的值为 2.



ALU 运算结果和数据输出为 2.

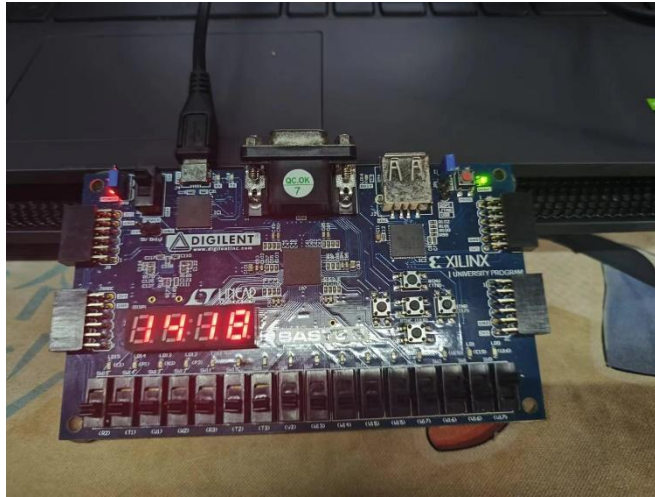


结果写入 5 号寄存器，5 号寄存器的值为 2.

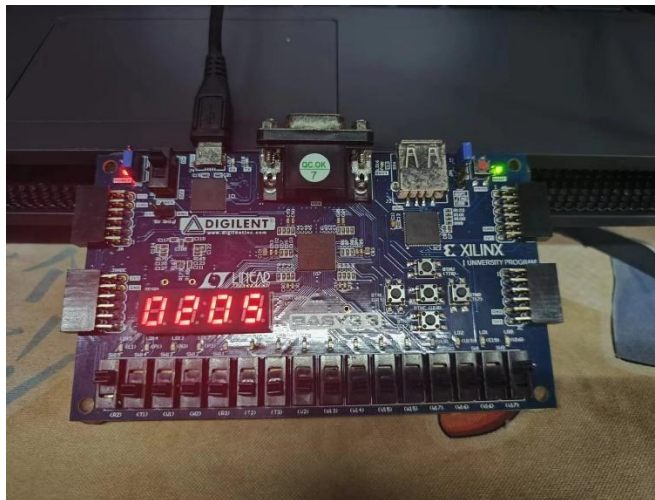


第六条指令: `sll $5,$5,2`

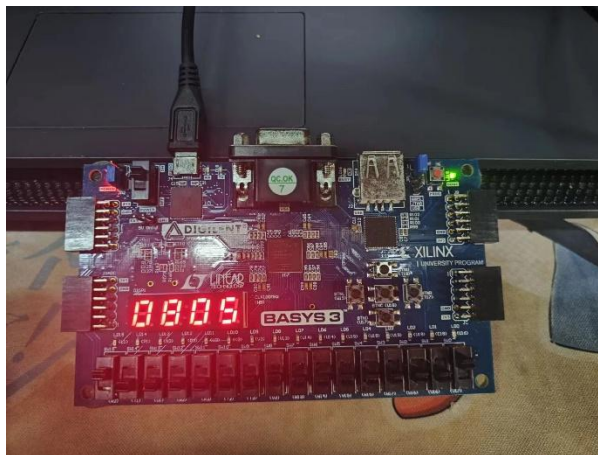
当前指令地址: 14, 下一条指令地址: 18



5 号寄存器的值为 2.



ALU 运算结果为 8，写入 5 号寄存器，5 号寄存器值为 8。



六、心得体会

通过实现单周期 CPU，我已经积累了一些经验，因此实现多周期 CPU 的过程相对顺利，但还是遇到了一些困难。

多周期 CPU 存在状态转换的问题，因此 ControlUnit 的设计更加复杂。在控制单元内，需要考虑不同状态控制信号不同的问题，以及控制信号能否及时转换的问题。在我的设计中，由于控制信号在时钟下降沿更新，指令在时钟上升沿更新，因此控制信号的更新

必须在指令更新之前，也就是说指令需要在上一条指令的最后一个状态进行到一半时更新，才能保证 CPU 正常工作。这个问题困扰了我很久，最终通过看仿真图的控制信号和状态，一一对应，发现了问题，解决了问题。

在多周期 CPU 实现时（即烧板时），我还遇到了一个问题。我的板子在执行完 00000018 和 0000001C 指令之后，不会再次回到 00000018 指令。经过检查，我发现，第一条指令的运行结果并没有写入寄存器中，进而发现是第一条指令没有被读取导致的。我对 debunce 模块进行了修改，让 signal 从下降沿开始，这样就可以读取第一条指令了。

实现多周期 CPU，增强了我遇到问题时追本溯源、通过仿真波形和板子上的数据来分析问题原因，并解决问题的能力。